

Spectral Embeddings as Initialization for t-SNE

Walter Brookby

May 23, 2026

Abstract

This thesis examines the mathematical structure underlying the t-distributed Stochastic Neighbor Embedding (t-SNE) algorithm, with a particular emphasis on understanding how different initialization schemes influence the optimization landscape and final embeddings. The central theoretical contribution of the thesis is an analysis of the “small-scale limit” of the Stochastic Neighbor Embedding (SNE) objective. We prove that, under an appropriate scaling regime, the SNE cost function is approximated by a quadratic energy of the form $\sum_{i,j} p_{ij} \|y_i - y_j\|^2$. Consequently, the scaled SNE problem can be approximated by a spectral optimization problem whose solutions are given by eigenmaps. This establishes a precise mathematical connection between eigenmaps and t-SNE. We further show that another popular method of dimension reduction, Uniform Manifold Approximation and Projection (UMAP), does not admit an analogous spectral formulation. This explains why Laplacian-based initializations are meaningful for t-SNE but not for UMAP. Finally, we present empirical comparisons of PCA, spectral, and Laplacian eigenmaps initializations for t-SNE across several datasets. We analyze convergence behavior, embedding stability, and final KL divergence, demonstrating that spectral initializations typically yield faster optimization and improved global organization. Taken together, these results provide a clearer mathematical understanding of the role of initialization in t-SNE and highlight the value of spectral methods in nonlinear dimensionality reduction.

Contents

1	Introduction	3
2	Mathematical Background	4
2.1	Linear Algebra	4
2.2	Principal Component Analysis	8
2.3	Entropy and Relative Entropy	12
2.4	Taylor's Theorem	14
2.5	The Mathematics of Gradient Descent	15
2.6	Laplace Eigenmaps	18
3	The t-SNE Algorithm	19
3.1	High- and low-dimensional similarities	20
3.2	Cost function	21
4	Spectral Problem	23
4.1	Scaled t-SNE and the Spectral Problem	24
4.2	Main approximation	26
4.3	Interpretation	29
5	Experimental Results	29
5.1	Baseline t-SNE Behavior on MNIST	30
5.2	Effect of Initialization with Gradient Descent	31
5.3	Initialization Geometry Without Gradient Descent	32
5.4	Cost Function Comparisons	32
5.5	Computational Cost	35
5.6	Additional Datasets	35
5.7	Discussion: Theory vs Practice	38
6	The spectral problem in UMAP	38
6.1	UMAP's cost function	38
7	Conclusion	39
7.1	Limitations	39
7.2	Broader Implications	40
	References	41

1 Introduction

High-dimensional datasets arise across modern science, from imaging and genomics to finance and social networks. One widely used approach to visualize this data is t-distributed Stochastic Neighbor Embedding (t-SNE), a statistical method designed to map high-dimensional data into two or three dimensions.

t-SNE begins by defining a probability distribution over pairs of points in the high-dimensional space, where the probabilities reflect similarity between points. The goal is to construct a low-dimensional representation in which nearby points correspond to high-probability pairs, while dissimilar points are mapped farther apart. This is accomplished through gradient descent, minimizing the Kullback–Leibler divergence between the similarity distribution in the high-dimensional space and its counterpart in the embedding. In this way, t-SNE produces low-dimensional visualizations that emphasize local neighborhood structure and reveal clusters within the data.

One of the central challenges in applying t-SNE is that the convergence of gradient descent depends heavily on the initial conditions. The standard initialization method is Principal Component Analysis (PCA), a linear technique that preserves global variance but does not always maintain the geometric structure of the data. This thesis introduces a spectral initialization method, which we call the Spectral Embedding, derived from the eigenvectors of the matrix $D^{(p)} - P$ where $P = (p_{ij})$ encodes the high-dimensional affinities and $D^{(p)}$ is its associated degree matrix. This construction closely resembles the classical Laplacian Eigenmaps method of Belkin and Niyogi [1], which similarly computes low-dimensional coordinates from the eigenvectors of a graph Laplacian, but differs in that our spectral problem arises directly from a quadratic approximation of the t-SNE objective rather than being imposed as an independent embedding criterion. We analyze t-SNE both mathematically and visually and compare its divergence-based formulation with the spectral objectives underlying Eigenmaps. The results show that spectral initialization can improve the interpretability and consistency of t-SNE embeddings, highlighting deeper mathematical connections between spectral graph theory and probabilistic divergence methods. We evaluate these results through a series of visualizations and by comparing the cost functions across multiple trials of Eigenmaps-based initialization and classical PCA initialization.

We validate these theoretical findings empirically across a diverse suite of datasets, ranging from structured image benchmarks to biological and tabular data, comparing PCA and spectral initializations in terms of convergence behavior, embedding stability, and final KL divergence.

We also briefly discuss UMAP, a popular nonlinear dimension reduction method. Unlike t-SNE, UMAP’s binary cross-entropy objective does not approximate a Laplacian quadratic form, so spectral initialization is not theoretically

justified in that setting. Although classical UMAP includes an option for a Laplacian Eigenmaps initialization, it is not well-motivated theoretically and lies outside the scope of this project.

2 Mathematical Background

This section reviews the mathematical tools that underlie all subsequent constructions in this thesis. Our primary focus is linear algebra and spectral theory, which provide a principled framework for identifying low-dimensional structure in high-dimensional data.

We begin with symmetric matrices and the Spectral Theorem, which guarantee the existence of orthogonal eigenvector bases for the covariance matrices and affinity matrices that appear throughout the thesis. We then introduce positive semidefiniteness, a property that ensures non-negative spectra and enables variational characterizations of eigenvalues. Finally, we discuss the Rayleigh Quotient, which unifies Principal Component Analysis (PCA) and spectral embedding methods as solutions to constrained optimization problems.

These results will be used repeatedly in later sections to analyze dimensionality reduction techniques and to motivate the use of Eigenmaps as an initialization strategy for the nonconvex optimization problem underlying t-SNE.

2.1 Linear Algebra

To analyze the transformation of data from a high-dimensional ambient space $\mathbb{R}^d$ to a lower-dimensional latent space $\mathbb{R}^s$, we rely on classical results from matrix theory. These results make it possible to decompose complex interactions between data points into orthogonal components that can be ordered by their relative importance.

2.1.1 Symmetric Matrices and the Spectral Theorem

A matrix $M \in \mathbb{R}^{n \times n}$ is said to be symmetric if $M = M^\top$. Many matrices of interest in this thesis are symmetric by construction, including the covariance matrix used in PCA and the matrix $D^{(p)} - P$ used in the Spectral Embedding.

The Spectral Theorem states that any real symmetric matrix is orthogonally diagonalizable. That is, there exists an orthonormal matrix Q (satisfying $Q^\top Q = I$) and a diagonal matrix Λ such that

$$M = Q\Lambda Q^\top.$$

The diagonal entries of Λ are the eigenvalues of M , and the columns of Q are the corresponding orthonormal eigenvectors.

This decomposition allows us to express quadratic forms, energy functionals, and optimization objectives in a coordinate system where each direction contributes independently. In dimensionality reduction, these eigenvectors define directions along which variance or smoothness is extremized.

We work over $\mathbb{R}$ and equip $\mathbb{R}^n$ with the standard inner product $\langle x, y \rangle = x^\top y$ and norm $\|x\|^2 = \langle x, x \rangle$.

Theorem 2.1 (Spectral Theorem for Real Symmetric Matrices). *Let $M \in \mathbb{R}^{n \times n}$ be symmetric, i.e. $M = M^\top$. Then there exist real numbers $\lambda_1, \dots, \lambda_n$ and an orthonormal basis $v_1, \dots, v_n$ of $\mathbb{R}^n$ such that*

$$Mv_i = \lambda_i v_i \quad \text{for each } i = 1, \dots, n.$$

Equivalently, letting $Q = [v_1 \ \cdots \ v_n]$ and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$, we have

$$M = Q\Lambda Q^\top,$$

where Q is orthogonal ($Q^\top Q = I$).

The proof proceeds in three steps: (i) we show that a symmetric matrix has at least one real eigenpair, obtained by maximizing a quadratic form on the unit sphere; (ii) we show that the orthogonal complement of an eigenvector is invariant under M ; and (iii) we apply induction on dimension.

Lemma 2.2 (Self-adjointness identity). *If $M = M^\top$, then for all $x, y \in \mathbb{R}^n$,*

$$\langle Mx, y \rangle = \langle x, My \rangle.$$

Proof. Using the definition of the inner product and symmetry of M ,

$$\langle Mx, y \rangle = (Mx)^\top y = x^\top M^\top y = x^\top My = \langle x, My \rangle.$$

□

Lemma 2.3 (Existence of an eigenvalue via the Rayleigh quotient). *Let $M = M^\top$ and define the Rayleigh quotient*

$$R(x) = \frac{x^\top Mx}{x^\top x}, \quad x \neq 0.$$

Then R attains a maximum on the unit sphere $S^{n-1} = \{x : \|x\| = 1\}$. Moreover, any maximizer $v \in S^{n-1}$ satisfies $Mv = \lambda v$ for $\lambda = R(v)$. In particular, M has a real eigenvalue.

Proof. Consider the function $f(x) = x^\top Mx$ restricted to S^{n-1} . Since f is continuous and S^{n-1} is compact, f attains a maximum at some $v \in S^{n-1}$.

To characterize v , we maximize $f(x)$ subject to the constraint $g(x) = x^\top x = 1$. By the method of Lagrange multipliers, there exists $\lambda \in \mathbb{R}$ such that

$$\nabla f(v) = \lambda \nabla g(v).$$

A direct calculation gives $\nabla f(x) = (M + M^\top)x = 2Mx$, since $M = M^\top$, and $\nabla g(x) = 2x$. Therefore $2Mv = \lambda(2v)$, i.e. $Mv = \lambda v$.

Finally, since $\|v\| = 1$, we have $\lambda = v^\top Mv = R(v)$, so λ is an eigenvalue of M . $\square$

Lemma 2.4 (Orthogonality of eigenvectors with distinct eigenvalues). *Let $M = M^\top$. If $Mv = \lambda v$ and $Mw = \mu w$ with $\lambda \neq \mu$, then $v \perp w$.*

Proof. Using the self-adjointness identity,

$$\lambda \langle v, w \rangle = \langle \lambda v, w \rangle = \langle Mv, w \rangle = \langle v, Mw \rangle = \langle v, \mu w \rangle = \mu \langle v, w \rangle.$$

Thus $(\lambda - \mu) \langle v, w \rangle = 0$, and since $\lambda \neq \mu$, we conclude $\langle v, w \rangle = 0$. $\square$

Lemma 2.5 (Invariant orthogonal complement). *Let $M = M^\top$ and let v be an eigenvector of M with eigenvalue λ . Then the orthogonal complement*

$$W := v^\perp = \{x \in \mathbb{R}^n : \langle x, v \rangle = 0\}$$

is invariant under M , i.e. $M(W) \subseteq W$.

Proof. Let $x \in W$, so $\langle x, v \rangle = 0$. Then, using self-adjointness and $Mv = \lambda v$,

$$\langle Mx, v \rangle = \langle x, Mv \rangle = \langle x, \lambda v \rangle = \lambda \langle x, v \rangle = 0.$$

Hence $Mx \in v^\perp = W$. This proves invariance. $\square$

Proof of the Spectral Theorem. We proceed by induction on $n = \dim(\mathbb{R}^n)$.

Base case: If $n = 1$, the result is immediate: any 1×1 symmetric matrix is $[\lambda]$ and $v_1 = (1)$ is an eigenvector.

Inductive step: Assume the theorem holds for dimension $n - 1$. Let $M \in \mathbb{R}^{n \times n}$ be symmetric. By the existence lemma above, M has a real eigenvalue λ_1 with a unit eigenvector v_1 .

Let $W = v_1^\perp$, which has dimension $n - 1$. By the invariant orthogonal complement lemma, W is M -invariant. Let $M_W : W \rightarrow W$ denote the restriction of M to W . The restriction is still self-adjoint on W , because for $x, y \in W$,

$$\langle M_W x, y \rangle = \langle Mx, y \rangle = \langle x, My \rangle = \langle x, M_W y \rangle.$$

Thus M_W is represented (in an orthonormal basis of W) by a symmetric $(n-1) \times (n-1)$ matrix.

By the inductive hypothesis applied to M_W , there exists an orthonormal basis $v_2, \dots, v_n$ of W consisting of eigenvectors of M_W , hence of M . Together, $v_1, \dots, v_n$ form an orthonormal basis of $\mathbb{R}^n$ and each satisfies $Mv_i = \lambda_i v_i$ for some real λ_i .

Finally, let $Q = [v_1 \cdots v_n]$ and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$. Then Q is orthogonal and $MQ = Q\Lambda$, so multiplying by $Q^\top$ gives $M = Q\Lambda Q^\top$. $\square$

2.1.2 Cauchy-Schwarz inequality

The Cauchy-Schwarz inequality is a basic but powerful result in linear algebra that relates inner products to vector lengths. In this thesis, the Cauchy-Schwarz inequality is used as a key tool for controlling and approximating gradient expressions and is central to the quadratic approximation of the SNE objective derived in Section 4.

Theorem 2.6 (Cauchy-Schwarz Inequality). *Let $(V, \langle \cdot, \cdot \rangle)$ be an inner product space over $\mathbb{R}$. Then for all $x, y \in V$,*

$$|\langle x, y \rangle| \leq \|x\| \|y\|,$$

with equality if and only if x and y are linearly dependent (i.e. $x = \lambda y$ for some scalar λ), including the case that one of them is 0.

Proof. If $y = 0$, then $\langle x, y \rangle = 0$ and the inequality is immediate. Thus, assume that $y \neq 0$. Then for any scalar $\alpha \in \mathbb{R}$, consider

$$0 \leq \|x - \alpha y\|^2 = \langle x - \alpha y, x - \alpha y \rangle.$$

Expanding using bilinearity of the inner product,

$$\|x - \alpha y\|^2 = \langle x, x \rangle - 2\alpha \langle x, y \rangle + \alpha^2 \langle y, y \rangle.$$

Choose

$$\alpha = \frac{\langle x, y \rangle}{\langle y, y \rangle}.$$

Substituting gives

$$0 \leq \|x - \alpha y\|^2 = \|x\|^2 - \frac{\langle x, y \rangle^2}{\|y\|^2}.$$

Rearranging yields

$$\langle x, y \rangle^2 \leq \|x\|^2 \|y\|^2,$$

and taking square roots, since both sides are non-negative, gives

$$|\langle x, y \rangle| \leq \|x\| \|y\|.$$

Equality holds if and only if $\|x - \alpha y\|^2 = 0$, i.e. $x - \alpha y = 0$, i.e. $x = \alpha y$. Thus, equality occurs exactly when x and y are linearly dependent, or one is zero. $\square$

2.1.3 The Rayleigh Quotient

Many spectral methods can be expressed as optimization problems involving the Rayleigh Quotient,

$$R(M, v) = \frac{v^\top M v}{\|v\|^2}.$$

The Rayleigh–Ritz principle, stated below, characterizes eigenvalues as extrema of this quotient.

As we will see in Sections 2.2 and 2.6, maximizing the Rayleigh quotient of the covariance matrix and minimizing the Rayleigh quotient of the graph Laplacian in spectral embeddings underlie the two families of dimensionality reduction methods studied in this thesis.

Theorem 2.7 (Rayleigh–Ritz Principle). *Let M be a symmetric matrix with eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$. Then for all nonzero $x \in \mathbb{R}^n$,*

$$\lambda_n \leq R(M, x) \leq \lambda_1,$$

with equality if and only if x is an eigenvector corresponding to the extremal eigenvalue.

Proof. Let $\{v_i\}$ be an orthonormal eigenbasis of M , and write $x = \sum_i c_i v_i$. Then,

$$x^\top M x = \sum_i \lambda_i c_i^2, \quad x^\top x = \sum_i c_i^2.$$

Dividing yields

$$R(M, x) = \frac{\sum_i \lambda_i c_i^2}{\sum_i c_i^2},$$

which is a weighted average of the eigenvalues. The bounds follow immediately. $\square$

2.2 Principal Component Analysis

Principal Component Analysis (PCA) is a classical linear method for reducing the dimensionality of a dataset while preserving as much variability as possible. The central idea is to identify a collection of orthogonal directions, called principal components, along which the data exhibit the greatest variance. Projecting the data onto the leading components yields a lower-dimensional representation that captures the dominant structure of the dataset.

2.2.1 Data Centering and the Covariance Matrix

Let $X \in \mathbb{R}^{n \times d}$ denote a dataset consisting of n samples in $\mathbb{R}^d$, where each row corresponds to a data point. PCA begins by centering the dataset: we subtract the mean

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i$$

from each sample, such that the transformed data satisfies

$$\frac{1}{n} \sum_{i=1}^n (x_i - \mu) = 0.$$

Centering ensures that variance computations reflect how points vary around the mean rather than the origin. The covariance matrix of the centered data is

$$C = \frac{1}{n} X^\top X \in \mathbb{R}^{d \times d}.$$

The entry C_{ij} measures the covariance between the i th and j th coordinates across the dataset. Intuitively, C captures the shape of the data cloud in $\mathbb{R}^d$: directions with large variance correspond to directions in which C stretches the most.

2.2.2 Eigenvalues, Eigenvectors, and Directions of Maximal Variance

A central fact used by PCA is that the eigenvectors of the covariance matrix C identify the orthogonal directions of maximal variance. If $v \in \mathbb{R}^d$ is a unit vector, the variance of the data when projected onto v is

$$\text{Var}(Xv) = v^\top C v.$$

Lemma 2.8 (Rayleigh Quotient). *For a symmetric matrix C , the Rayleigh quotient*

$$R(v) = \frac{v^\top C v}{\|v\|^2}$$

is maximized over nonzero v by the eigenvector corresponding to the largest eigenvalue of C .

Proof. Because C is symmetric, it admits an orthonormal eigenbasis:

$$C = Q \Lambda Q^\top, \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_d).$$

Let $v \in \mathbb{R}^d$ with $\|v\| = 1$, and set $x = Q^\top v$. Since Q is orthogonal, x also satisfies $\|x\| = 1$. Then

$$v^\top C v = v^\top Q \Lambda Q^\top v = x^\top \Lambda x = \sum_{i=1}^d \lambda_i x_i^2.$$

The coefficients x_i^2 satisfy $x_i^2 \geq 0$ and $\sum_i x_i^2 = 1$. Thus

$$v^\top C v = \sum_{i=1}^d \lambda_i x_i^2 \leq \lambda_1 \sum_{i=1}^d x_i^2 = \lambda_1,$$

with equality if and only if $x_1 = \pm 1$ and all other x_i vanish, i.e., $v = \pm q_1$, the eigenvector for λ_1 . The minimum is obtained similarly by using λ_d . $\square$

By the lemma, the solution to the optimization problem is the eigenvector v_1 associated with the largest eigenvalue λ_1 . More generally, if

$$C v_k = \lambda_k v_k, \quad \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d \geq 0,$$

then v_k represents the direction of the k th-largest variance, subject to orthogonality. The eigenvectors $v_1, \dots, v_d$ are the principal axes, and the eigenvalues λ_k give the variance captured by each axis.

2.2.3 Dimensionality Reduction and Reconstruction Error

Let $X \in \mathbb{R}^{n \times d}$ be a data matrix whose rows are centered data points $x_i \in \mathbb{R}^d$, i.e.,

$$\frac{1}{n} \sum_{i=1}^n x_i = 0.$$

To reduce the dimensionality from d to $s \ll d$, PCA selects the directions along which the data vary the most. These directions are given by the eigenvectors of the covariance matrix

$$C = \frac{1}{n} X^\top X.$$

Let

$$C v_k = \lambda_k v_k, \quad \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d \geq 0.$$

The eigenvectors corresponding to the largest eigenvalues represent the directions of greatest variance in the dataset. Choosing the first s eigenvectors defines the principal component subspace. Let

$$V_s = [v_1 \ v_2 \ \dots \ v_s] \in \mathbb{R}^{d \times s}$$

be the matrix whose columns are these principal components. The vectors $v_1, \dots, v_s$ are orthonormal, meaning they are mutually orthogonal and each has unit length.

Projecting the data onto this s -dimensional subspace gives the reduced representation

$$Y = X V_s \in \mathbb{R}^{n \times s}.$$

Each row of Y therefore contains the coordinates of the original data point expressed in the principal component basis. The original data can be approximately reconstructed from this lower-dimensional representation by mapping the points back into $\mathbb{R}^d$:

$$\hat{X} = YV_s^\top = XV_sV_s^\top.$$

Because the projection discards some information, the reconstructed data $\hat{X}$ will generally differ from the original data X . The difference

$$X - XV_sV_s^\top$$

is called the reconstruction error. To measure the total size of this error we use the Frobenius norm, defined for a matrix A by

$$\|A\|_F^2 = \sum_{i,j} A_{ij}^2,$$

which is simply the sum of the squares of all entries of the matrix.

Theorem 2.9 (Optimality of PCA Subspace). *Let $C = \frac{1}{n}X^\top X$ be the sample covariance matrix of the centered data. Among all s -dimensional subspaces, the one minimizing the reconstruction error*

$$\|X - XV_sV_s^\top\|_F^2$$

is spanned by the eigenvectors corresponding to the s largest eigenvalues of C .

Proof. For any orthonormal matrix V_s , the reconstruction error is

$$\|X - XV_sV_s^\top\|_F^2 = \|X\|_F^2 - \|XV_s\|_F^2.$$

Since $\|X\|_F^2$ is constant, minimizing the error is equivalent to maximizing

$$\|XV_s\|_F^2 = \text{tr}(V_s^\top X^\top XV_s) = n \text{tr}(V_s^\top CV_s).$$

By the Rayleigh–Ritz theorem, this trace is maximized when the columns of V_s are the eigenvectors corresponding to the s largest eigenvalues of C . $\square$

Thus, PCA yields the optimal rank- s linear approximation of the dataset. Equivalently, if $X = U\Sigma W^\top$ is the singular value decomposition (SVD), the principal components are given by the first s columns of W , and the reduced representation is $Y = U_s\Sigma_s$.

2.2.4 Role of PCA in This Project

PCA plays two distinct but related roles in this project. First, all datasets are initially reduced to their top 30 principal components, preserving global variance structure while suppressing high-dimensional noise. Second, PCA provides a natural initialization for t-SNE. Beginning the nonlinear optimization from a representation that already respects large-scale geometry improves stability and accelerates convergence. Subsequent optimization by t-SNE then refines the embedding to emphasize local neighborhood relationships.

2.3 Entropy and Relative Entropy

To understand the cost function optimized by t-SNE, we need the concepts of entropy and relative entropy. These ideas come from information theory, introduced by Claude Shannon in his foundational 1948 paper on communication theory [5], and form the mathematical basis for measuring the similarity between probability distributions. t-SNE relies on these quantities to compare neighborhoods in high- and low-dimensional space.

2.3.1 Shannon Entropy

Let $P = (p_1, \dots, p_n)$ be a discrete probability distribution. The Shannon entropy of P is defined by

$$H(P) = - \sum_{i=1}^n p_i \log_2 p_i.$$

Entropy measures the amount of uncertainty or randomness in the distribution: a sharply peaked distribution has low entropy, while a uniform distribution has maximal entropy. In the context of dimension reduction, entropy helps quantify how concentrated a point's neighborhood is in high-dimensional space. We use logarithms base 2, as is standard in information theory; the base is omitted in subsequent formulas since it only introduces a constant scaling.

2.3.2 Relative Entropy (Kullback–Leibler Divergence)

Let $P = (p_i)$ and $Q = (q_i)$ be probability distributions on a finite set $\{1, \dots, m\}$, meaning $p_i \geq 0$, $q_i \geq 0$, and

$$\sum_{i=1}^m p_i = \sum_{i=1}^m q_i = 1.$$

The relative entropy, or Kullback–Leibler divergence, from P to Q is defined by

$$D_{\text{KL}}(P||Q) = \sum_{i=1}^m p_i \log \frac{p_i}{q_i},$$

with the conventions that $0 \log(0) = 0$ and that $D_{\text{KL}}(P\|Q) = +\infty$ if $p_i > 0$ but $q_i = 0$ for some i .

Proposition 2.10 (Nonnegativity of KL Divergence). *For probability distributions P and Q ,*

$$D_{\text{KL}}(P\|Q) \geq 0,$$

with equality if and only if $p_i = q_i$ for all i .

Proof. Applying the inequality $\log x \leq x - 1$ (for $x > 0$) with $x = q_i/p_i$ gives

$$\log \frac{q_i}{p_i} \leq \frac{q_i}{p_i} - 1,$$

or equivalently,

$$-\log \frac{p_i}{q_i} \leq \frac{q_i}{p_i} - 1.$$

Multiplying by $p_i \geq 0$ and summing over i gives

$$-\sum_{i=1}^m p_i \log \frac{p_i}{q_i} \leq \sum_{i=1}^m q_i - \sum_{i=1}^m p_i = 1 - 1 = 0,$$

and therefore $D_{\text{KL}}(P\|Q) = \sum_{i=1}^m p_i \log \frac{p_i}{q_i} \geq 0$. Equality holds if and only if $q_i/p_i = 1$ for all i , which is equivalent to $p_i = q_i$ for all i . $\square$

KL divergence therefore provides a nonnegative measure of how different two probability distributions are.

2.3.3 Entropy and KL Divergence in t-SNE

t-SNE constructs a probability distribution $P = (p_{ij})$ in the high-dimensional space, where p_{ij} measures the similarity between points x_i and x_j . The indices satisfy $i, j \in \{1, \dots, n\}$, for $i \neq j$, and the probabilities are normalized so that

$$\sum_{i \neq j} p_{ij} = 1.$$

Given a low-dimensional embedding $Y = (y_1, \dots, y_n)$, t-SNE defines a second distribution $Q = (q_{ij}(Y))$ that measures similarities between embedded points y_i and y_j . The t-SNE cost function is the KL divergence between these two distributions:

$$C(Y) = D_{\text{KL}}(P\|Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}(Y)}.$$

Minimizing this objective encourages the low-dimensional embedding to match the neighborhood relationships encoded by the high-dimensional distribution P .

2.4 Taylor's Theorem

In this section, we provide a rigorous proof of Taylor's Theorem with the Lagrange form of the remainder. This result is fundamental to justifying the quadratic approximation of the SNE cost function used in Section 4.

2.4.1 Taylor's Theorem with Lagrange Remainder

Theorem 2.11 (Taylor's Theorem). *Let $f : [a, b] \rightarrow \mathbb{R}$ be a function such that f is $(n + 1)$ -times differentiable and $f^{(n)}$ is continuous on $[a, b]$ and $f^{(n+1)}$ exists on (a, b) . Let $x_0 \in [a, b]$. For any $x \in [a, b]$, there exists a point ξ strictly between x_0 and x such that*

$$f(x) = \sum_{k=0}^n \frac{f^{(k)}(x_0)}{k!} (x - x_0)^k + R_n(x)$$

where the Lagrange Remainder term $R_n(x)$ is given by

$$R_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} (x - x_0)^{n+1}.$$

Proof. Fix $x \in [a, b]$ with $x \neq x_0$. We define a constant M such that

$$f(x) = \sum_{k=0}^n \frac{f^{(k)}(x_0)}{k!} (x - x_0)^k + M(x - x_0)^{n+1}.$$

Now, consider the auxiliary function $g(t)$ defined by

$$g(t) = f(t) - \sum_{k=0}^n \frac{f^{(k)}(x_0)}{k!} (t - x_0)^k - M(t - x_0)^{n+1}.$$

By the properties of the Taylor polynomial, we observe that the derivatives of g at x_0 satisfy $g(x_0) = g'(x_0) = g''(x_0) = \dots = g^{(n)}(x_0) = 0$. Furthermore, by our choice of M , we have $g(x) = 0$.

Since $g(x_0) = 0$ and $g(x) = 0$, Rolle's Theorem implies there exists c_1 between x_0 and x such that $g'(c_1) = 0$. Given $g'(x_0) = 0$ and $g'(c_1) = 0$, applying Rolle's Theorem again implies there exists c_2 between x_0 and c_1 such that $g''(c_2) = 0$.

By repeating this process $n + 1$ times, we conclude there exists a point $\xi = c_{n+1}$ between x_0 and x such that $g^{(n+1)}(\xi) = 0$. Differentiating $g(t)$ exactly $n + 1$ times yields

$$g^{(n+1)}(t) = f^{(n+1)}(t) - M(n+1)!.$$

Evaluating at $t = \xi$, we obtain $0 = f^{(n+1)}(\xi) - M(n+1)!$, which implies $M = \frac{f^{(n+1)}(\xi)}{(n+1)!}$. Substituting this back into our expression for $f(x)$ completes the proof. $\square$

2.4.2 Derivation of Scalar Bounds for SNE

Using the case where $n = 1$, we derive the specific inequalities that will be applied in Section 4 to approximate the Scaled SNE objective.

Let $f(x) = e^{-x}$ with $x_0 = 0$. By Taylor's Theorem, $e^{-x} = 1 - x + \frac{e^{-\xi}}{2}x^2$ for some $\xi \in (0, x)$. Since $e^{-\xi} \leq 1$ for all $\xi > 0$, it follows that

$$|e^{-x} - (1 - x)| \leq x^2 \quad x > 0.$$

Let $f(x) = \log(1 - x)$ with $x_0 = 0$. Taylor's Theorem gives $\log(1 - x) = -x - \frac{1}{2(1-\xi)^2}x^2$ for some $\xi \in (0, x)$. For x in a small neighborhood of the origin (e.g., $x \in [0, 1/2]$), the term $\frac{1}{2(1-\xi)^2}$ is bounded by a constant c , justifying the bound

$$|\log(1 - x) + x| \leq x^2 \quad x \in [0, \frac{1}{2}].$$

These bounds will be used in Section 4 to control the error terms arising in the quadratic approximation of the scaled SNE objective.

2.5 The Mathematics of Gradient Descent

Gradient descent is a first-order optimization algorithm used to minimize a differentiable cost function by iteratively moving in the direction of steepest decrease. Suppose we wish to minimize a function

$$C(\theta) : \mathbb{R}^d \rightarrow \mathbb{R}.$$

The key idea is that the gradient $\nabla C(\theta)$ points in the direction of steepest increase of C , so moving in the opposite direction decreases the value of the cost most rapidly.

Theorem 2.12 (Gradient Gives the Direction of Steepest Ascent). *Let $C : \mathbb{R}^d \rightarrow \mathbb{R}$ be differentiable. For any unit vector u , the directional derivative of C at θ in direction u is*

$$D_u C(\theta) = \nabla C(\theta)^\top u.$$

This quantity is maximized when

$$u = \frac{\nabla C(\theta)}{\|\nabla C(\theta)\|}.$$

Proof. By the Cauchy-Schwarz inequality,

$$\nabla C(\theta)^\top u \leq \|\nabla C(\theta)\| \cdot \|u\| = \|\nabla C(\theta)\|.$$

Equality holds exactly when u is proportional to $\nabla C(\theta)$. Since u must have unit norm, the maximizer is $u = \nabla C(\theta) / \|\nabla C(\theta)\|$. $\square$

Formally, by Taylor expanding C around a point θ ,

$$C(\theta + \Delta) \approx C(\theta) + \nabla C(\theta)^\top \Delta,$$

so choosing $\Delta = -\eta \nabla C(\theta)$ decreases the cost as rapidly as possible at first order for a chosen step size $\eta > 0$. This yields the classical gradient descent update:

$$\theta_{t+1} = \theta_t - \eta \nabla C(\theta_t).$$

Lemma 2.13 (First-Order Decrease of Gradient Descent). *Let $C : \mathbb{R}^d \rightarrow \mathbb{R}$ be differentiable. For sufficiently small $\eta > 0$,*

$$C(\theta - \eta \nabla C(\theta)) = C(\theta) - \eta \|\nabla C(\theta)\|^2 + o(\eta).$$

Thus, taking a small step in the negative gradient direction decreases the cost.

Proof. By the first-order Taylor expansion,

$$C(\theta - \eta \nabla C(\theta)) = C(\theta) - \eta \nabla C(\theta)^\top \nabla C(\theta) + o(\eta) = C(\theta) - \eta \|\nabla C(\theta)\|^2 + o(\eta).$$

For small η , the second term is negative unless $\nabla C(\theta) = 0$, showing the cost decreases. $\square$

The scalar η is called the learning rate, and it controls the magnitude of each step. If η is too large, the algorithm may overshoot regions of low cost or diverge entirely; if too small, convergence becomes extremely slow. In high-dimensional and highly non-linear cost landscapes, such as those encountered in t-SNE, the choice of η strongly affects both stability and the quality of the final solution.

2.5.1 Gradient Descent Applied to t-SNE

In t-SNE, the parameters being optimized are the low-dimensional embedding coordinates

$$Y = (y_1, \dots, y_n) \in \mathbb{R}^{n \times d},$$

where typically $d = 2$ or 3 . The cost function is the Kullback–Leibler (KL) divergence between the high-dimensional similarity distribution $P = (p_{ij})$ and the low-dimensional similarity distribution $Q = (q_{ij})$:

$$C(Y) = \text{KL}(P \parallel Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}(Y)}.$$

Because q_{ij} depends on the pairwise distances $\|y_i - y_j\|^2$, the gradient of the KL divergence with respect to each point y_i can be explicitly computed. Differentiation yields the t-SNE force equation (shown in section 3.2):

$$\frac{\partial C}{\partial y_i} = 4 \sum_{j \neq m} \frac{(p_{ij} - q_{ij})(y_i - y_j)}{1 + \|y_m - y_j\|^2}.$$

This expression naturally separates into attractive forces proportional to p_{ij} , which pull similar points together, and repulsive forces proportional to q_{ij} , which push all points apart. The gradient descent update for each embedding point takes the form

$$y_i^{t+1} = y_i^t - \eta \frac{\partial C}{\partial y_i}.$$

Because the forces may vary greatly in magnitude, the dynamics can be unstable unless additional techniques are introduced.

2.5.2 Momentum and Early Exaggeration

To stabilize the updates, t-SNE includes a momentum term, which accumulates past gradients to smooth the trajectory:

$$\Delta y_i^t = \alpha \Delta y_i^{t-1} - \eta \frac{\partial C}{\partial y_i^t}, \quad y_i^{t+1} = y_i^t + \Delta y_i^t.$$

Here $\alpha \in [0, 1)$ controls how much previous updates influence the current step. Momentum helps the algorithm escape shallow local minima and reduces oscillations.

A second important modification is the early exaggeration phase. During early iterations, t-SNE uses early exaggeration:

$$p_{ij}^{(\text{exag})} = \tau p_{ij}, \quad \tau > 1,$$

temporarily strengthening attractive forces, encouraging tight cluster formation before repulsive forces spread clusters globally. Geometrically, this widens gaps between clusters and encourages more meaningful organization of the embedding.

2.5.3 Initialization and Non-Convexity of the t-SNE Landscape

The cost surface of t-SNE is highly non-convex, with many local minima arising from the complex interplay between attractive and repulsive forces. As a consequence, the final embedding can vary significantly depending on the initialization of Y .

Common initializations include random Gaussian initialization, which provides no structural information about the data, PCA initialization, which preserves global linear variance, and spectral or Laplacian Eigenmaps initialization, which is derived from the eigenvectors of a matrix encoding pairwise affinities p_{ij} . Because spectral and Laplacian initializations are aligned with the affinity structure of the data, they place the optimizer in a region of the cost surface closer to meaningful minima, leading to faster convergence and fewer

instabilities than PCA or random initialization. The theoretical justification for this is developed in Section 4.

In summary, the iterative updates of gradient descent, combined with momentum, early exaggeration, and a geometrically informed initialization, form the core mathematical machinery that enables t-SNE to refine the embedding Y .

2.6 Laplace Eigenmaps

Laplacian Eigenmaps, introduced by Belkin and Niyogi [1], provide a spectral method for nonlinear dimensionality reduction. The key idea is to represent the dataset as a weighted graph that captures local neighborhood relationships between data points and then compute a low-dimensional embedding using the eigenvectors of a graph Laplacian.

2.6.1 Graph Construction

Let $x_1, \dots, x_n \in \mathbb{R}^d$ be a dataset. We construct a weighted graph $G = (V, E)$ with vertices $V = \{1, \dots, n\}$, where each vertex corresponds to a data point. Edges are assigned weights that measure similarity between points. A common choice is the Gaussian kernel

$$w_{ij} = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right),$$

for nearby points, with $w_{ij} = 0$ for pairs that are not considered neighbors. Let $W = (w_{ij})$ denote the resulting weight matrix.

Define the diagonal degree matrix D by

$$D_{ii} = \sum_{j=1}^n w_{ij}.$$

The matrix

$$L = D - W$$

is called the (unnormalized) graph Laplacian.

2.6.2 Spectral Embedding

The Laplacian Eigenmaps embedding is obtained by solving the optimization problem

$$\min_{y_1, \dots, y_n \in \mathbb{R}^s} \sum_{i,j} w_{ij} \|y_i - y_j\|^2,$$

subject to the normalization constraints

$$Y^\top DY = I, \quad Y^\top D\mathbf{1} = 0,$$

where $Y = (y_1, \dots, y_n)^\top \in \mathbb{R}^{n \times s}$ and $\mathbf{1}$ denotes the constant vector.

Intuitively, this objective encourages points that are strongly connected in the graph (large w_{ij}) to remain close together in the embedding. A direct calculation shows that

$$\sum_{i,j} w_{ij} \|y_i - y_j\|^2 = 2 \operatorname{tr}(Y^\top LY).$$

Thus, the problem reduces to minimizing a quadratic form involving the graph Laplacian.

Theorem 2.14. *Let $L = D - W$ be the graph Laplacian. The solution to the Laplacian Eigenmaps optimization problem is given by the eigenvectors corresponding to the smallest nonzero eigenvalues of the generalized eigenvalue problem*

$$Lv = \lambda Dv.$$

The smallest eigenvalue is $\lambda_1 = 0$, with eigenvector $\mathbf{1}$. Discarding this trivial solution, the next s eigenvectors provide coordinates for the embedding.

2.6.3 Relation to Spectral Methods

Laplacian Eigenmaps produce a low-dimensional representation by computing eigenvectors of a matrix derived from pairwise affinities between data points. This spectral formulation closely resembles the quadratic energy that will arise later from the small-scale approximation of the t-SNE objective. In that setting, the matrix $D(p) - P$ plays a role analogous to a graph Laplacian, leading naturally to eigenvector-based coordinates for the embedding.

3 The t-SNE Algorithm

Let $x_1, \dots, x_n \in \mathbb{R}^d$ denote n data points in high-dimensional space. The goal of t-SNE is to find low-dimensional embeddings $y_1, \dots, y_n \in \mathbb{R}^s$, where $s \ll d$ is usually $s = 2$ or 3 , that preserve neighborhood structure. A central idea of the algorithm is that direct comparison of the distance between $\mathbb{R}^d$ and $\mathbb{R}^s$ is unreliable, as high-dimensional spaces distances tend to concentrate, while in low dimensions distances are forced into a limited volume. To overcome this, t-SNE represents neighborhoods not by distances but by probability distributions. The algorithm constructs a probability distribution $P = (p_{ij})$ in the high-dimensional space and another distribution $Q = (q_{ij})$ in the embedding space, and then seeks an embedding for which these two distributions are as close as possible in the sense of Kullback–Leibler divergence. We also assume that $n > s + 1$, otherwise we could find an s -plane in $\mathbb{R}^d$ containing all the points. The algorithm proceeds as follows:

3.1 High- and low-dimensional similarities

In the high-dimensional space, t-SNE models the neighborhood around a point $x_i \in R^d$ using a Gaussian distribution centered at x_i . For each pair $i \neq j$ we define conditional probabilities

$$p_{j|i} = \frac{e^{\left(-\frac{\|x_i - x_j\|^2}{2\sigma_i^2}\right)}}{\sum_{k \neq i} e^{\left(-\frac{\|x_i - x_k\|^2}{2\sigma_i^2}\right)}}, i \neq j,$$

which measure the likelihood that x_i would select x_j as a neighbor. The Gaussian kernel assigns high weight to points close to x_i and rapidly decays for distant points, producing an affinity structure that reflects the local geometry of the dataset. The variance σ_i^2 is chosen adaptively so that the Shannon entropy of the conditional distribution $p_{j|i}$ matches a user-specified perplexity, which determines the effective number of neighbors that each point should consider. Because the conditional distribution is not symmetric, t-SNE defines a symmetric joint distribution by setting

$$p_{ij} = \frac{p_{i|j} + p_{j|i}}{2n}, i \neq j$$

and $p_{ii} = 0$.

Proposition 3.1. $P = (p_{ij})$ is a probability distribution.

Proof. To show that P is a probability distribution, we must show that $\sum p_{ij} = 1$ and that each $p_{ij} \geq 0$. Since $p_{ij} = \frac{p_{i|j} + p_{j|i}}{2n}$ and $p_{i|j}$ is symmetric and positive, as the exponential is always positive. Therefore $p_{ij} \geq 0$. Additionally,

$$\begin{aligned} \sum p_{ij} &= \sum \frac{p_{i|j} + p_{j|i}}{2n} = \frac{1}{2n} \left(\sum p_{i|j} + \sum p_{j|i} \right) \\ &= \frac{1}{2n} \left(\sum_{i=1}^n \sum_{j \neq i} \frac{e^{\left(-\frac{\|x_i - x_j\|^2}{2\sigma_i^2}\right)}}{\sum_{k \neq i} e^{\left(-\frac{\|x_i - x_k\|^2}{2\sigma_i^2}\right)}} + \sum_{i=1}^n \sum_{j \neq i} \frac{e^{\left(-\frac{\|x_j - x_i\|^2}{2\sigma_j^2}\right)}}{\sum_{k \neq j} e^{\left(-\frac{\|x_j - x_k\|^2}{2\sigma_j^2}\right)}} \right) \\ &= \frac{1}{2n} \sum_{i=1}^n 1 + \sum_{i=1}^n 1 = \frac{1}{2n} (n + n) = 1 \end{aligned}$$

□

Intuitively, p_{ij} represents the probability that i and j are mutual neighbors in the high-dimensional space. p_{ij} defines a probability distribution over all unordered pairs, with larger p_{ij} corresponding to greater similarity in the high-dimensional space.

In the low dimensional embedding $Y = (y_1, \dots, y_n)$, where $y_i \in R^s$, define

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}}, i \neq j.$$

This expression uses the kernel $(1 + r^2)^{-1}$, which corresponds to a Student's t-distribution with one degree of freedom, also known as a Cauchy distribution. The choice of a heavy-tailed distribution is essential. If a Gaussian kernel were used in the low-dimensional space, all moderately distant points would be assigned nearly zero similarity, causing clusters to collapse too tightly and producing severe distortions known as the ‘‘crowding problem.’’ The one-degree-of-freedom t-distribution provides the heaviest possible tails while still being normalizable, ensuring that distant points retain a small but meaningful influence on the embedding dynamics. The result is that the distribution Q more accurately reflects the geometric relationships that can be realized in a low-dimensional embedding.

Proposition 3.2. $Q = (q_{ij})$ is a probability distribution.

Proof. To show that Q is a probability distribution, we must show that $\sum q_{ij} = 1$ and that each $q_{ij} \geq 0$. Since $q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}}$ and $(1 + \|y_i - y_j\|^2) > 0$ for all y_i, y_j . Therefore each $q_{ij} \geq 0$. Additionally,

$$\sum q_{ij} = \sum_{i \neq j} \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}} = 1$$

□

The purpose of defining these two distributions is to make the geometry of the dataset comparable across dimensions. Distances in high and low dimensions behave very differently, especially because high-dimensional spaces exhibit phenomena such as concentration of measure and distorted neighborhoods. Probability distributions avoid these issues by allowing t-SNE to compare the relative similarities of points rather than their raw distances. The cost function between P and Q then quantifies how much neighborhood information is lost when moving from the high-dimensional space to the embedding.

3.2 Cost function

The embeddings are chosen to minimize the Kullback-Leibler (KL) divergence between the high- and low-dimensional similarity distributions:

$$C(Y) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}.$$

The asymmetry of KL divergence matters. Particularly, if a pair has high p_{ij} (close neighbors in the original space) but low q_{ij} (too far apart in the embedding), the cost increases sharply. Conversely, large q_{ij} values for pairs with small p_{ij} are penalized only mildly. This asymmetry is what makes t-SNE prioritize preserving local neighborhoods rather than global geometry.

This functional is nonnegative and vanishes if and only if $p_{ij} = q_{ij}$. In practice, minimizing $C(Y)$ is carried out using gradient descent with momentum and various heuristics, such as "early exaggeration."

Proposition 3.3 (Gradient of the t-SNE objective). *Assume the symmetric t-SNE setting with affinities $p_{ij} = p_{ji} \geq 0$ and $q_{ij} = q_{ji}$ satisfying $\sum_{i \neq j} p_{ij} = \sum_{i \neq j} q_{ij} = 1$.*

The objective $C(Y) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$ has gradient for each $m \in \{1, \dots, n\}$:

$$\nabla_{y_m} C = 4 \sum_{j \neq m} \frac{(p_{mj} - q_{mj})(y_m - y_j)}{1 + \|y_m - y_j\|^2}. \quad (3.1)$$

Proof. We will begin with our cost function

$$\begin{aligned} C &= \sum_{i \neq j} p_{ij} \log\left(\frac{p_{ij}}{q_{ij}}\right) \\ &= \sum_{i \neq j} p_{ij} \log(p_{ij}) - \sum_{i \neq j} p_{ij} \log(q_{ij}). \end{aligned}$$

Where,

$$q_{ij} = \frac{w_{ij}}{V}, \quad w_{ij} = (1 + \|y_i - y_j\|^2)^{-1}, \quad V = \sum_{a \neq b} w_{ab}.$$

We can ignore the first term as it does not depend on y , thus

$$C = - \sum_{i \neq j} p_{ij} \log(q_{ij}).$$

Thus, the gradient of C using the chain rule is,

$$\nabla_{y_m} C = - \sum_{i \neq j} p_{ij} \frac{1}{w_{ij}} \nabla w_{ij} + \left(\sum_{i \neq j} p_{ij} \right) \left(\frac{1}{V} \nabla V \right).$$

Considering the first term,

$$\begin{aligned} - \sum_{i \neq j} p_{ij} \frac{1}{w_{ij}} \nabla w_{ij} &= - \sum_{i \neq j} p_{ij} (1 + \|y_i - y_j\|^2) \frac{2(y_i - y_j)}{(1 + \|y_i - y_j\|^2)^2} \\ &= - \sum_{i \neq j} p_{ij} \frac{2(y_i - y_j)}{(1 + \|y_i - y_j\|^2)}. \end{aligned}$$

Because the only terms that contribute are $m = i, j$,

$$\sum_{i \neq j} p_{ij} \frac{2(y_i - y_j)}{(1 + \|y_i - y_j\|^2)} = - \sum_{j \neq m} p_{mj} \frac{2(y_m - y_j)}{(1 + \|y_m - y_j\|^2)} - \sum_{i \neq m} p_{im} \frac{2(y_i - y_m)}{(1 + \|y_i - y_m\|^2)}.$$

Now, we can show that $\sum_{i \neq m} p_{im} \frac{2(y_i - y_m)}{(1 + \|y_i - y_m\|^2)} = \sum_{j \neq m} p_{mj} \frac{2(y_m - y_j)}{(1 + \|y_m - y_j\|^2)}$, since $p_{im} = p_{mj}$. Thus, the first term is,

$$\sum_{j \neq m} \frac{4p_{mj}}{1 + \|y_m - y_j\|^2} (y_m - y_j).$$

The second term follows from the same logic as simplifying the first term. It is important to note, $q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{V}$ and thus, $\frac{(1 + \|y_i - y_j\|^2)^{-1}}{q_{ij}} = V$. Therefore,

$$\begin{aligned} \left(\sum_{i \neq j} p_{ij} \right) \left(\frac{1}{V} \nabla V \right) &= \sum_{i \neq j} (1 + \|y_i - y_j\|^2) \left(\sum_{i \neq j} \frac{-4(y_i - y_j)}{(1 + \|y_i - y_j\|^2)^2} \right) \\ &= \sum_{i \neq j} \frac{-4(y_i - y_j)}{1 + \|y_i - y_j\|^2} q_{ij}. \end{aligned}$$

Thus, combining our first and second terms,

$$\nabla C = 4 \sum_{m \neq j} \left(\frac{(y_m - y_j)}{1 + \|y_m - y_j\|^2} \right) (p_{mj} - q_{mj}).$$

□

The gradient shows how points are pulled together or pushed apart depending on whether their low-dimensional similarity q_{mj} under- or overestimates the high-dimensional similarity p_{mj} .

4 Spectral Problem

While t-SNE is formulated through probabilistic similarities and KL divergence, we show that its objective admits a structured quadratic approximation in the small-scale regime. In this regime, the dominant contribution to the objective arises from pairwise squared distances weighted by the fixed high-dimensional affinities p_{ij} . As shown below, minimizing this quadratic energy leads naturally to an eigenvalue problem for a symmetric matrix derived directly from $P = [p_{ij}]$.

Although similar quadratic energies appear in spectral graph methods, the spectral problem studied here is not introduced as an independent graph-based model. Rather, it emerges as a consequence of a second-order approximation of the scaled t-SNE objective. The resulting eigenvectors provide low-dimensional coordinates that capture the leading geometric structure encoded by the affinities p_{ij} .

4.1 Scaled t-SNE and the Spectral Problem

We recall t-SNE minimizes, for $Y = [y_1, \dots, y_n] \in \mathbb{R}^s$, the cost function

$$C(Y) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}, \quad q_{ij} = \frac{e^{-\|y_i - y_j\|^2}}{\sum_{k \neq \ell} e^{-\|y_k - y_\ell\|^2}},$$

with fixed $p_{ij} > 0$ for $i \neq j$ and $\sum_{i \neq j} p_{ij} = 1$. We fix the overall position and size of the embedding by requiring it to be centered at the origin and to have a fixed total variance.

For a small parameter $\delta > 0$ and ambient dimension s , define

$$Y^\top Y = \frac{(n-1)\delta}{2s} I_s, \quad \sum_{i=1}^n y_i = 0. \quad (4.1)$$

Where I_s is the $s \times s$ identity matrix. Taking traces in (4.1) gives

$$\text{tr}(Y^\top Y) = \sum_{i=1}^n \|y_i\|^2 = \frac{n-1}{2} \delta.$$

Thus, the embedding is centered at the origin and scaled so that its total squared radius equals $\frac{(n-1)\delta}{2}$. Then,

$$\sum_{i=1}^n \|y_i\|^2 = \frac{n-1}{2} \delta, \quad \sum_{i=1}^n y_i = 0. \quad (4.2)$$

Lemma 4.1. *If $\sum_i y_i = 0$, then*

$$\sum_{i \neq j} \|y_i - y_j\|^2 = 2n \sum_{i=1}^n \|y_i\|^2.$$

Proof. Expand and use symmetry:

$$\sum_{i \neq j} \|y_i - y_j\|^2 = \sum_{i \neq j} (\|y_i\|^2 + \|y_j\|^2 - 2y_i^\top y_j) = 2(n-1) \sum_i \|y_i\|^2 - 2 \sum_{i \neq j} y_i^\top y_j.$$

But $\sum_{i \neq j} y_i^\top y_j = (\sum_i y_i)^\top (\sum_j y_j) - \sum_i \|y_i\|^2 = -\sum_i \|y_i\|^2$ since $\sum_i y_i = 0$. Hence the claim. $\square$

Combining (4.2) and Lemma 4.1 yields the equivalent constraints

$$\sum_{i \neq j} \|y_i - y_j\|^2 = n(n-1) \delta, \quad \sum_{i=1}^n y_i = 0. \quad (4.3)$$

Remark 4.2 (Heine–Borel Theorem). A subset $S \subset \mathbb{R}^m$ is compact if and only if it is closed and bounded.

The constraint functions in (4.3) are continuous, hence their joint level set is closed. By Lemma 4.1, $\sum_i \|y_i\|^2 = (n-1)\delta/2$, so all embeddings in $\mathcal{Y}_\delta$ lie within a bounded Euclidean ball in $(\mathbb{R}^s)^n$. By the Heine–Borel theorem, $\mathcal{Y}_\delta$ is therefore compact. Let $\mathcal{Y}_\delta$ denote the compact set of Y obeying (4.3). Throughout we assume $\delta \ll 1/n^2$.

4.1.1 Elementary analytic bounds

We will repeatedly use the following Taylor-type inequalities.

Lemma 4.3 (Scalar bounds). *There exists a universal constant $c > 0$ such that for all $x \in [0, c]$,*

$$\left| e^{-x} - (1 - x) \right| \leq x^2, \quad \left| \log(1 - x) + x \right| \leq x^2.$$

Proof. For $f(x) = e^{-x}$, Taylor's theorem with Lagrange remainder gives $e^{-x} = 1 - x + \frac{1}{2}e^{-\xi}x^2$ for some $\xi \in (0, x)$, hence $|e^{-x} - (1 - x)| \leq \frac{1}{2}x^2 \leq x^2$ on $[0, 1]$. For $g(x) = \log(1 - x)$, Taylor's theorem at 0 yields $\log(1 - x) = -x - \frac{x^2}{2(1-\zeta)^2}$ for some $\zeta \in (0, x)$. Thus $|\log(1 - x) + x| \leq \frac{x^2}{2(1-x)^2}$, which is $\leq x^2$ on $[0, 1/2]$. Take $c = 1/2$. $\square$

4.1.2 Decomposition of the SNE objective

Write $Z(Y) := \sum_{k \neq \ell} e^{-\|y_k - y_\ell\|^2}$. Then

$$\begin{aligned} C(Y) &= \sum_{i \neq j} p_{ij} \log p_{ij} - \sum_{i \neq j} p_{ij} \log q_{ij} \\ &= \sum_{i \neq j} p_{ij} \log p_{ij} + \sum_{i \neq j} p_{ij} \|y_i - y_j\|^2 + \sum_{i \neq j} p_{ij} \log Z(Y). \end{aligned} \quad (4.4)$$

Because $\sum_{i \neq j} p_{ij} = 1$, the last sum equals $\log Z(Y)$. We now approximate $Z(Y)$ using Lemma 4.3. Define for each pair (k, ℓ)

$$E_{k\ell} := e^{-\|y_k - y_\ell\|^2} - \left(1 - \|y_k - y_\ell\|^2\right), \quad \text{so that } |E_{k\ell}| \leq \|y_k - y_\ell\|^4.$$

Summing and using (4.3) gives

$$Z(Y) = \sum_{k \neq \ell} \left(1 - \|y_k - y_\ell\|^2\right) + \sum_{k \neq \ell} E_{k\ell} = n(n-1)(1 - \delta) + \sum_{k \neq \ell} E_{k\ell}. \quad (4.5)$$

Lemma 4.4. *Under the constraints (4.3) and $\delta \ll 1/n^2$, we have*

$$\left| \sum_{k \neq \ell} E_{k\ell} \right| = O\left(n^2(n-1)^2\delta^2\right).$$

Proof. Recall that $E_{k\ell} := e^{-\|y_k - y_\ell\|^2} - (1 - \|y_k - y_\ell\|^2)$, and that by Lemma 4.3, $|E_{k\ell}| \leq \|y_k - y_\ell\|^4$ for all pairs (k, ℓ) , provided $\|y_k - y_\ell\|^2 \leq c$. Since $\delta \ll 1/n^2$, constraint (4.1) gives $\sum_{k \neq \ell} \|y_k - y_\ell\|^2 = n(n-1)\delta \ll 1$, so each individual term satisfies $\|y_k - y_\ell\|^2 \leq n(n-1)\delta \ll 1 \leq c$, and the pointwise bound applies.

Since every term is nonnegative, no individual term can exceed the total sum, so for every pair (k, ℓ) ,

$$\|y_k - y_\ell\|^2 \leq \sum_{k \neq \ell} \|y_k - y_\ell\|^2 = n(n-1)\delta = O(n(n-1)\delta).$$

Therefore,

$$\begin{aligned} \sum_{k \neq \ell} \|y_k - y_\ell\|^4 &\leq O(n(n-1)\delta) \cdot \sum_{k \neq \ell} \|y_k - y_\ell\|^2 \\ &= O(n(n-1)\delta) \cdot n(n-1)\delta \\ &= O(n^2(n-1)^2\delta^2). \end{aligned}$$

Since $|E_{k\ell}| \leq \|y_k - y_\ell\|^4$, summing over all pairs gives

$$\left| \sum_{k \neq \ell} E_{k\ell} \right| \leq \sum_{k \neq \ell} \|y_k - y_\ell\|^4 = O(n^2(n-1)^2\delta^2).$$

□

By Lemma 4.3 with $x = \delta - \frac{\varepsilon}{n(n-1)}$, noting that $|x| = O(\delta)$ is small under the assumption $\delta \ll \frac{1}{n^2}$,

$$\begin{aligned} \log Z(Y) &= \log \left(n(n-1) \left(1 - \delta + \frac{\varepsilon}{n(n-1)} \right) \right) \\ &= \log n(n-1) + \log \left(1 - \left(\delta - \frac{\varepsilon}{n(n-1)} \right) \right) \\ &= \log n(n-1) - \left(\delta - \frac{\varepsilon}{n(n-1)} \right) + O \left(\left(\delta - \frac{\varepsilon}{n(n-1)} \right)^2 \right) \\ &= \log n(n-1) - \delta + O(n(n-1)\delta^2) \end{aligned} \tag{4.6}$$

4.2 Main approximation

Plug (4.6) into (4.4) to obtain

$$C(Y) = \sum_{i \neq j} p_{ij} \log p_{ij} + \sum_{i \neq j} p_{ij} \|y_i - y_j\|^2 + \log n(n-1) - \delta + O(n(n-1)\delta^2). \tag{4.7}$$

Therefore, over the constraint set $\mathcal{Y}_\delta$,

$$C(Y) = \text{const} + \sum_{i \neq j} p_{ij} \|y_i - y_j\|^2 + O(n(n-1)\delta^2),$$

where “const” does not depend on Y .

Theorem 4.5 (Scaled SNE $\rightarrow$ quadratic energy). *Suppose $\delta \ll 1/n^2$. Then*

$$\inf_{Y \in \mathcal{Y}_\delta} C(Y) = \text{const} + \inf_{Y \in \mathcal{Y}_\delta} \sum_{i \neq j} p_{ij} \|y_i - y_j\|^2 + O(n(n-1)\delta^2).$$

Consequently, any Y minimizing the quadratic energy on $\mathcal{Y}_\delta$ is an $O(n(n-1)\delta^2)$ -approximate minimizer of the SNE objective on $\mathcal{Y}_\delta$.

Proof. From (4.7), for every $Y \in \mathcal{Y}_\delta$ we can write

$$C(Y) = \text{const} + E(Y) + R(Y), \quad E(Y) := \sum_{i \neq j} p_{ij} \|y_i - y_j\|^2,$$

where const is independent of Y and

$$|R(Y)| \leq K n(n-1)\delta^2 \quad \text{for all } Y \in \mathcal{Y}_\delta$$

for some constant $K > 0$.

Taking infima over $\mathcal{Y}_\delta$ yields

$$\inf_{\mathcal{Y}_\delta} C = \text{const} + \inf_{\mathcal{Y}_\delta} E + O(n(n-1)\delta^2).$$

If Y^* minimizes E on $\mathcal{Y}_\delta$, then for any $Y \in \mathcal{Y}_\delta$,

$$\begin{aligned} C(Y^*) &\leq \text{const} + E(Y^*) + K n(n-1)\delta^2 \\ &\leq \text{const} + E(Y) + K n(n-1)\delta^2 \\ &\leq C(Y) + 2K n(n-1)\delta^2. \end{aligned}$$

Taking $\inf_{Y \in \mathcal{Y}_\delta}$ on the right proves that Y^* is an $O(n(n-1)\delta^2)$ -approximate minimizer of C on $\mathcal{Y}_\delta$. $\square$

4.2.1 The eigenvalue problem for the quadratic SNE energy

Consider for all $s \geq 1$. We write $y_i = (Y_{i,1}, \dots, Y_{i,s}) \in \mathbb{R}^s$ for $i = 1, \dots, n$, and let $Y = (Y_{i,\alpha}) = (y_1, \dots, y_n)^\top \in \mathbb{R}^{n \times s}$. Recall that $p_{ii} = 0$ and define the diagonal matrix $D^{(p)}$ by $D_{ii}^{(p)} = \sum_{j \neq i} p_{ij}$. Then a direct expansion gives

$$\begin{aligned} \sum_{i \neq j} p_{ij} \|y_i - y_j\|^2 &= \sum_{i \neq j} p_{ij} (\|y_i\|^2 + \|y_j\|^2 - 2y_i \cdot y_j) \\ &= \sum_i D_{ii}^{(p)} \|y_i\|^2 + \sum_j D_{jj}^{(p)} \|y_j\|^2 - 2 \sum_{i \neq j} p_{ij} y_i \cdot y_j \\ &= 2 \sum_{i,j} (D^{(p)} - P)_{ij} y_i \cdot y_j. \end{aligned}$$

Writing this in component form yields

$$\sum_{i \neq j} p_{ij} \|y_i - y_j\|^2 = 2 \sum_{\alpha=1}^s \sum_{i,j=1}^n (D^{(p)} - P)_{ij} Y_{i\alpha} Y_{j\alpha} = 2 \operatorname{tr}(Y^\top (D^{(p)} - P) Y).$$

Let $\{v_1, \dots, v_n\}$ be an orthonormal eigenbasis of $D^{(p)} - P$ which we know exists by the Spectral Theorem, with $(D^{(p)} - P)v_k = \lambda_k v_k$, where $0 = \lambda_1 < \lambda_2 \leq \dots \leq \lambda_n$, and $v_1 \propto \mathbf{1}$. The centering constraint $\sum_{i=1}^n y_i = 0$ implies that each column vector $(Y_{1\alpha}, \dots, Y_{n\alpha})^\top$ is orthogonal to v_1 . Hence, for each $\alpha = 1, \dots, s$, we may write $(Y_{1\alpha}, \dots, Y_{n\alpha})^\top = \sum_{k=2}^n a_{k\alpha} v_k$ for some $a_{k\alpha} \in \mathbb{R}$.

Substituting this expansion into the quadratic form gives

$$\sum_{i \neq j} p_{ij} \|y_i - y_j\|^2 = 2 \sum_{\alpha=1}^s \sum_{k=2}^n \lambda_k a_{k\alpha}^2.$$

We now impose the normalization constraint $Y^\top Y = \frac{(n-1)\delta}{2s} I_s$. In terms of the coefficients $a_{k\alpha}$, this condition is equivalent to

$$\sum_{k=2}^n a_{k\alpha} a_{k\beta} = \frac{(n-1)\delta}{2s} \delta_{\alpha\beta}, \quad \alpha, \beta = 1, \dots, s, \quad (4.8)$$

which in particular implies that the coefficient vectors $a_\alpha := (a_{2\alpha}, \dots, a_{n\alpha}) \in \mathbb{R}^{n-1}$ are pairwise orthogonal and have equal norm. Under the constraint (4.8), the expression

$$2 \sum_{\alpha=1}^s \sum_{k=2}^n \lambda_k a_{k\alpha}^2$$

is minimized by concentrating each a_α along the eigenvectors corresponding to the smallest nonzero eigenvalues $\lambda_2, \dots, \lambda_{s+1}$. Consequently, the minimum is attained when

$$a_1 = \sqrt{\frac{(n-1)\delta}{2s}} e_1, \quad a_2 = \sqrt{\frac{(n-1)\delta}{2s}} e_2, \quad \dots, \quad a_s = \sqrt{\frac{(n-1)\delta}{2s}} e_s,$$

where $\{e_1, \dots, e_s\}$ is the standard basis of $\mathbb{R}^{n-1}$.

Equivalently, the minimizing embedding matrix is

$$Y = \sqrt{\frac{(n-1)\delta}{2s}} (v_2 \mid v_3 \mid \dots \mid v_{s+1}),$$

where $v_2, \dots, v_{s+1}$ are the eigenvectors of $D^{(p)} - P$ associated with the smallest nonzero eigenvalues.

4.3 Interpretation

The spectral coordinates y_i are constructed so that pairs of points with large SNE affinities p_{ij} remain close together in the embedding. Rather than explicitly optimizing a probabilistic divergence, the quadratic energy penalizes large distances between strongly coupled pairs, thereby encoding the local neighborhood structure induced by the high-dimensional similarities. The centering constraint $\sum_i y_i = 0$ removes the trivial constant mode, while the normalization constraint $Y^\top Y = \frac{(n-1)\delta}{2s} I_s$ fixes the overall scale of the embedding and prevents collapse.

4.3.1 Connection to t-SNE

Both t-SNE and the associated spectral problem aim to preserve local structure in the data, but they differ fundamentally in formulation and optimization. While t-SNE defines pairwise similarities p_{ij} and q_{ij} and minimizes the nonconvex Kullback–Leibler divergence $\text{KL}(P\|Q)$ using gradient-based optimization, our spectral problem arises from a quadratic approximation of the scaled t-SNE objective and reduces to minimizing the weighted energy

$$\sum_{i \neq j} p_{ij} \|y_i - y_j\|^2,$$

subject to centering and normalization constraints, yielding a closed-form solution via the smallest nontrivial eigenvectors of the symmetric matrix $D^{(p)} - P$.

From this perspective, the spectral problem is not an independent graph-based embedding method, but rather a structured approximation to t-SNE in the regime of small embedding scale. Although the resulting spectral embedding does not minimize the full t-SNE objective, it provides a geometrically meaningful starting point that aligns with the dominant quadratic component of the loss. We will demonstrate that in practice, this alignment yields more stable optimization dynamics and improved convergence when the spectral solution is used to initialize t-SNE, as we have just shown it does theoretically.

5 Experimental Results

Our analysis draws on a range of datasets that capture different types of high-dimensional structure. MNIST provides a large collection of handwritten digit images and serves as a standard benchmark for evaluating nonlinear embeddings with clear cluster patterns. The Olivetti Faces dataset and the COIL-20 object dataset introduce more complex geometric and photometric variation, offering settings where manifold structure exists but is less cleanly

separated. We also include a Kaggle credit card fraud dataset, which consists of mixed numerical features and presents a setting without an obvious underlying manifold. To explore representations derived from deep networks, we use feature embeddings extracted from a pretrained ResNet-18 model, which offer a high-dimensional space shaped by learned semantic structure. Finally, we analyze the PBMC3K single-cell RNA sequencing dataset, whose thousands of gene expression measurements per cell reflect the sparsity and heterogeneity typical of biological data. This diverse suite of datasets enables a broad comparison of PCA and spectral initializations across both structured and unstructured settings.

The MNIST dataset served as the primary experimental dataset throughout this project. While other datasets are used to assess robustness and generality, the main results are presented on MNIST, with supporting data from the additional five datasets discussed in Section 5.6. MNIST consists of grayscale images of handwritten digits and is widely used as a benchmark for dimensionality reduction algorithms. Its well-understood cluster structure and moderate size make it an ideal setting for studying the geometric and optimization properties of t-SNE in a controlled environment.

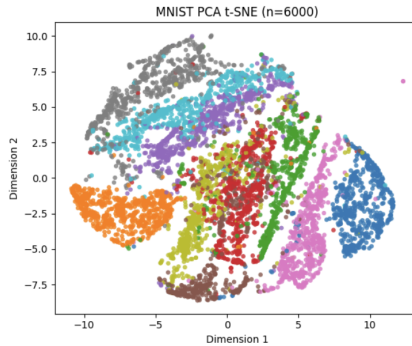


Figure 1: t-SNE embedding of 6,000 MNIST samples using a standard initialization and gradient descent. Distinct digit clusters emerge, illustrating the algorithm’s ability to preserve local neighborhood structure in a low-dimensional embedding.

5.1 Baseline t-SNE Behavior on MNIST

We begin by illustrating the qualitative behavior of t-SNE on MNIST using a standard PCA initialization and a sample size of 6,000 points. Figure 1 shows the resulting two-dimensional embedding after gradient descent. Distinct digit clusters emerge, demonstrating t-SNE’s ability to preserve local neighborhood structure while sacrificing global distances. At the same time, the irregular

global arrangement of clusters highlights the nonconvex nature of the t-SNE objective and foreshadows the sensitivity of the final embedding to initialization.

To contextualize the embedding results, Figure 2 displays representative MNIST images. Each data point corresponds to a 28×28 grayscale image, which is flattened into a high-dimensional vector prior to processing.

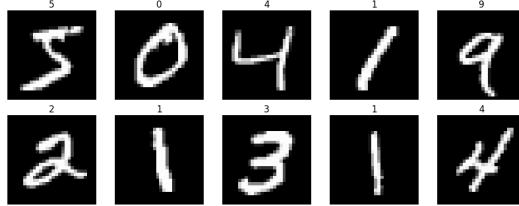


Figure 2: Representative MNIST digit images. Each image is treated as a point in $\mathbb{R}^{784}$ prior to dimensionality reduction.

5.2 Effect of Initialization with Gradient Descent

We next compare three initialization strategies for t-SNE: PCA initialization, spectral initialization, and Laplacian Eigenmap initialization. Figure 3 shows the resulting embeddings after gradient descent optimization.

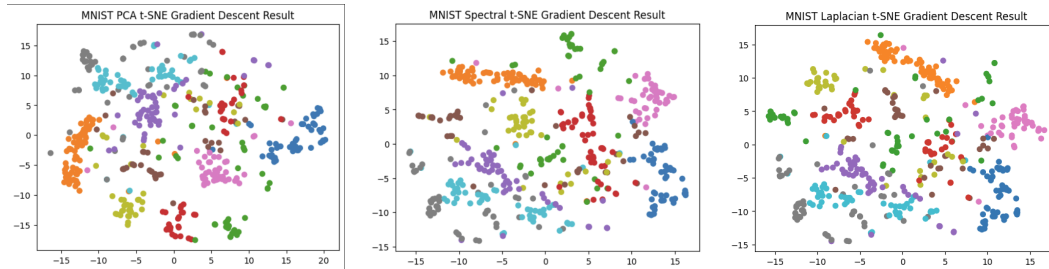


Figure 3: MNIST embeddings after gradient descent using different initialization strategies: PCA (left), spectral (center), and Laplacian (right). Each figure includes around 500 data points.

Although all three methods ultimately produce recognizable digit clusters, qualitative differences in cluster compactness, separation, and global organization persist. Spectral and Laplacian initializations tend to produce embeddings with more coherent global structure and fewer fragmented clusters, whereas PCA initialization exhibits greater variability in cluster placement and shape across runs. This suggests that while gradient descent partially mitigates differences between initializations, it does not fully erase the influence of the starting geometry.

5.3 Initialization Geometry Without Gradient Descent

Figure 4 isolates the geometric contribution of each initialization by visualizing the embeddings prior to any gradient-based optimization. Unlike PCA, which primarily preserves global variance without enforcing neighborhood smoothness, spectral and Laplacian initializations already exhibit meaningful local grouping and large-scale structure before optimization begins. In particular, clusters corresponding to similar digits appear partially separated, reflecting the influence of the graph Laplacian and its associated smoothness constraints.

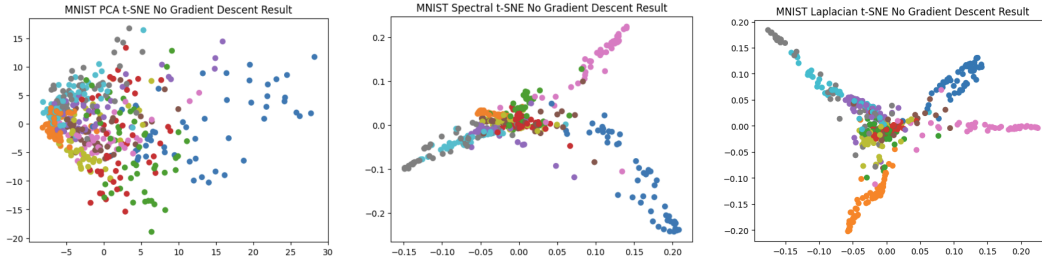


Figure 4: MNIST embeddings before gradient descent using different initialization strategies: PCA (left), spectral (center), and Laplacian (right).

This behavior is consistent with the theoretical analysis in Section 4, where we showed that in the small-scale limit, the t-SNE objective admits a quadratic approximation whose minimizers are given by Laplacian eigenmaps. From this perspective, spectral initialization can be understood as placing the optimizer near a low-energy region of the t-SNE landscape before gradient descent is applied. These plots reveal substantial differences in neighborhood structure and global arrangement, highlighting how different spectral constructions encode geometry even before optimization begins.

5.4 Cost Function Comparisons

To quantify the qualitative effects observed in the embeddings, we compare KL divergence across multiple independent runs using each initialization strategy. Figure 5 presents per-run KL divergence trajectories on MNIST, while Figure 6 provides a distributional summary via boxplots.

The pre-optimization results show a clear and consistent ordering. As shown in Figure 5a and 5b, PCA initialization yields substantially higher KL divergence at initialization than either spectral or Laplacian methods, with mean values of 2.9254 ± 0.0888 versus 2.6251 ± 0.0110 and 2.6154 ± 0.0102 respectively. The per-run line plot illustrates that PCA divergence fluctuates widely across samples, while the Spectral and Laplacian track closely together near a consistently lower value. The pairwise difference boxplots in Figure 6

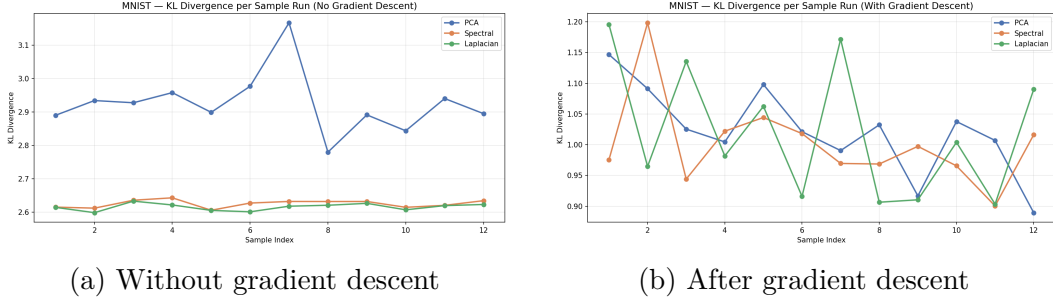


Figure 5: KL divergence across initialization methods on the MNIST dataset. Panel (a) shows cost trajectories without gradient descent, while panel (b) shows the corresponding trajectories after optimization.

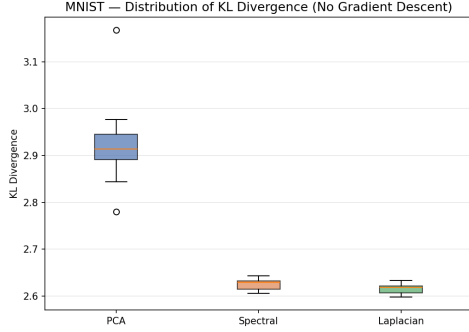
confirm this: differences of PCA–Spectral and PCA–Laplacian are tightly concentrated well above zero, while the Spectral–Laplacian differences are essentially zero, indicating that both spectral methods are nearly equivalent to one another and jointly superior to PCA as starting configurations.

These results directly validate the theoretical analysis of Section 4. Because the spectral initialization solves the quadratic approximation of the SNE objective, it begins in a region of the cost landscape that is already well-aligned with the dominant term of the t-SNE objective. PCA, which optimizes global variance rather than pairwise affinities, offers no such alignment.

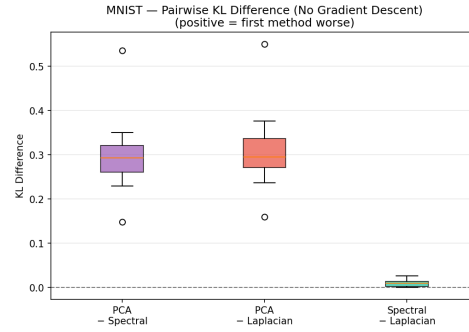
The post-optimization picture is more nuanced, and the gap between methods narrows considerably after gradient descent reduces the absolute KL divergence for all three methods. On MNIST, final mean KL values are 1.0217 ± 0.0687 (PCA), 1.0017 ± 0.0702 (Spectral), and 1.0202 ± 0.1033 (Laplacian). Spectral achieves the lowest mean, but the difference is modest, and the pairwise difference boxplots in Figure 6c show boxes straddling zero with substantial spread, indicating no single method dominates consistently across all runs.

This pattern generalizes across datasets, as shown in Figure 8. The best-performing method after gradient descent varies by dataset: Spectral performs best on MNIST and CIFAR-10, Laplacian on CreditCard and COIL-20, and PCA on PBMC.RNA and OlivettiFaces. No method achieves a statistically dominant advantage across all settings.

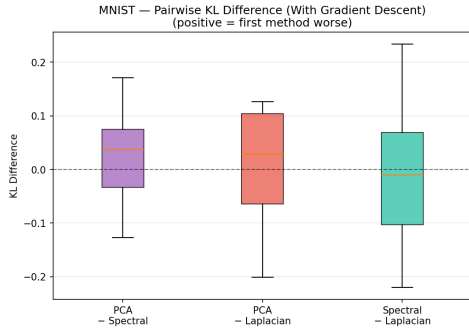
Taken together, these results suggest a refined interpretation of the role of initialization. The primary benefit of spectral initialization is not that it reliably achieves a lower final KL divergence after optimization, since gradient descent largely closes that gap regardless of starting point, but rather that it begins in a geometrically favorable region of the loss landscape, leading to more consistent convergence behavior and reduced variance across runs. This distinction is practically significant: a lower-variance initialization produces



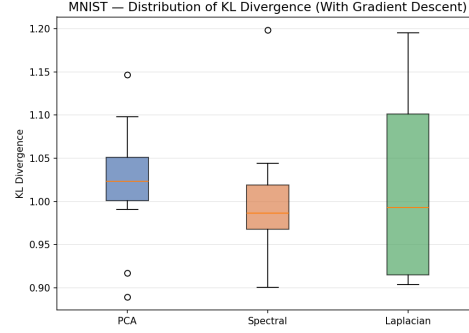
(a) Final KL divergence without gradient descent.



(b) Pairwise KL divergence differences without gradient descent.



(c) Pairwise KL divergence differences after gradient descent.



(d) Final KL divergence after gradient descent.

Figure 6: Distributional comparison of KL divergence outcomes across initialization strategies on the MNIST dataset. Panels (a) and (b) summarize results without gradient descent, while panels (c) and (d) show corresponding distributions after optimization.

more reproducible embeddings, which matters in scientific applications where embedding stability directly affects downstream conclusions.

To assess whether the behaviors observed on MNIST generalize beyond a toy dataset, we conducted analogous experiments on several additional datasets, including COIL-20, Olivetti Faces, and a real-world tabular dataset. Figure 8 provides a distributional view of these results.

5.5 Computational Cost

Initialization strategies introduce different computational overheads. Figure 7 compares average runtimes on MNIST across the three methods. Without gradient descent (Figure 7a), PCA is fastest at approximately 0.05s, while Spectral and Laplacian require 0.13s and 0.12s respectively, reflecting the additional cost of eigenvalue decomposition. However, once gradient descent is applied (Figure 7b), all three methods converge to essentially identical runtimes of approximately 28s, as the cost of optimization dominates the initialization overhead entirely. This implies that the geometric advantages of spectral initialization can be obtained at negligible additional cost in practice.

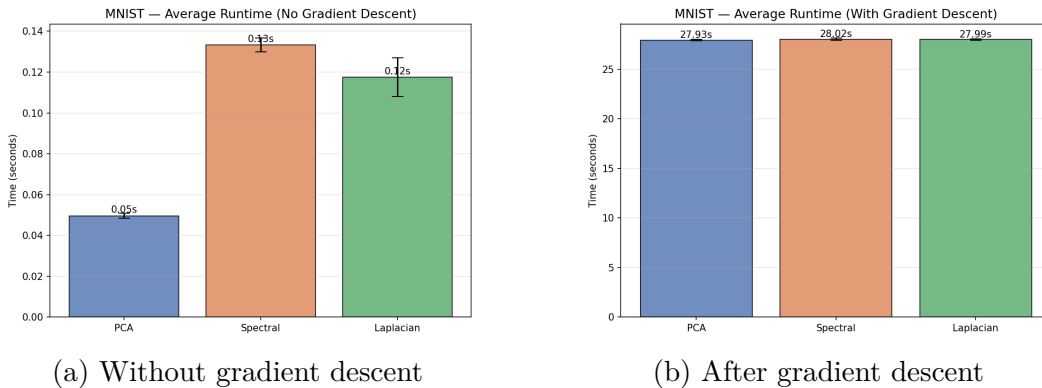


Figure 7: Average runtime across initialization strategies on the MNIST dataset. Panel (a) reports runtimes without gradient descent, while panel (b) reports runtimes after optimization.

5.6 Additional Datasets

Figure 8 summarizes initialization performance across all six datasets considered in this study. The pre-optimization results (Figure 8a) are consistent with the MNIST findings: PCA yields substantially higher KL divergence at initialization across nearly every dataset, while Spectral and Laplacian methods begin from lower-cost configurations. The post-optimization results are more mixed, with the best-performing method varying by dataset. Where the data admit clearer

manifold or graph structure, as with COIL-20 and PBMC3K, spectral methods tend to retain an advantage in the final embedding, while less structured data like the credit card dataset show more modest and inconsistent differences. This pattern is consistent with the theoretical expectation that spectral initialization is most beneficial when the data admit a meaningful affinity graph — precisely the setting in which the quadratic approximation of Section 4 is most accurate.

Mean KL Divergence $\pm$ Std — No Gradient Descent
(green = best per dataset)

	PCA	Spectral	Laplacian
MNIST	2.9254 $\pm$ 0.0888	2.6251 $\pm$ 0.0110	2.6154 $\pm$ 0.0102
CIFAR10	3.4051 $\pm$ 0.0292	2.6738 $\pm$ 0.0081	2.6948 $\pm$ 0.0083
CreditCard	3.5059 $\pm$ 0.1399	2.5765 $\pm$ 0.0137	2.5584 $\pm$ 0.0141
PBMC_RNA	4.1304 $\pm$ 0.0651	2.4300 $\pm$ 0.0127	2.4481 $\pm$ 0.0123
OlivettiFaces	1.4400 $\pm$ 0.0482	1.8110 $\pm$ 0.1405	1.7135 $\pm$ 0.0454
COIL20	2.1223 $\pm$ 0.0241	2.7464 $\pm$ 0.0055	2.7007 $\pm$ 0.0090

(a) Mean KL divergence before gradient descent.

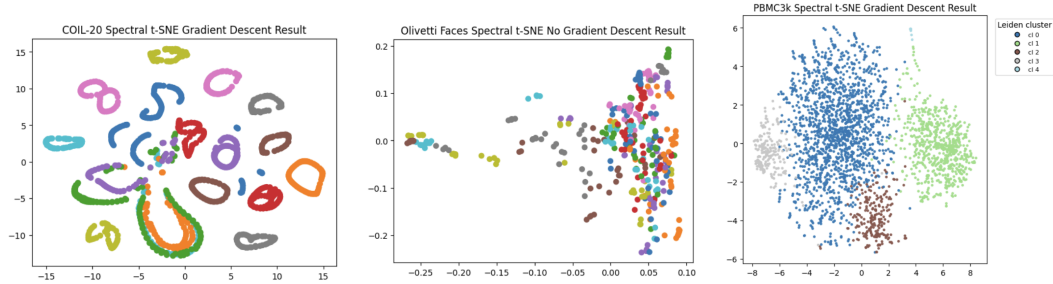
Mean KL Divergence $\pm$ Std — After Gradient Descent
(green = best per dataset)

	PCA	Spectral	Laplacian
MNIST	1.0217 $\pm$ 0.0687	1.0017 $\pm$ 0.0702	1.0202 $\pm$ 0.1033
CIFAR10	1.0075 $\pm$ 0.0393	1.0259 $\pm$ 0.0463	1.0024 $\pm$ 0.0462
CreditCard	1.2074 $\pm$ 0.0887	1.2112 $\pm$ 0.0613	1.2028 $\pm$ 0.0628
PBMC_RNA	1.4628 $\pm$ 0.0433	1.4844 $\pm$ 0.0384	1.4761 $\pm$ 0.0389
OlivettiFaces	0.5404 $\pm$ 0.1118	0.5595 $\pm$ 0.1317	0.5426 $\pm$ 0.0636
COIL20	0.2873 $\pm$ 0.0176	0.2973 $\pm$ 0.0254	0.2856 $\pm$ 0.0157

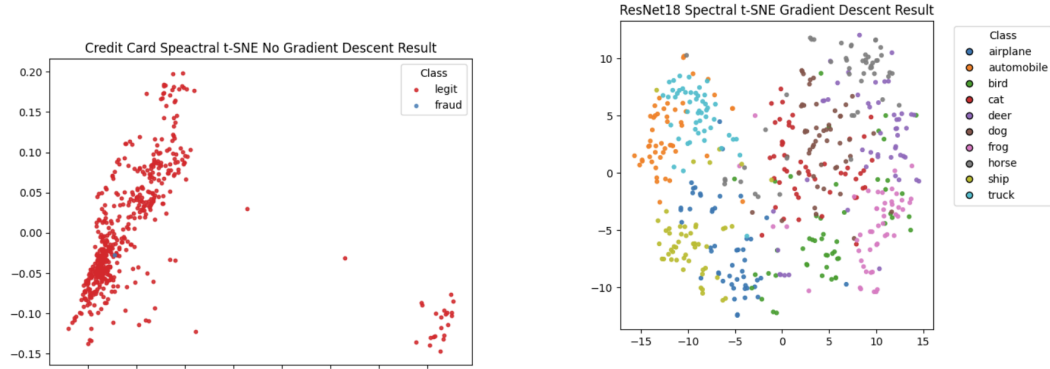
(b) Mean KL divergence after gradient descent.

Figure 8: Summary of initialization performance across all datasets. Green highlighting indicates the best-performing method per dataset.

Figure 9 presents representative t-SNE embeddings across multiple datasets using spectral initialization under different optimization settings. Although spectral initializations generally exhibit higher t-SNE cost before optimization, they nonetheless encode coherent local neighborhood structure that is visually apparent even before gradient descent. Subsequent optimization sharpens cluster boundaries and resolves global layout, with the magnitude of change varying substantially by dataset.



(a) COIL-20 dataset (spectral initialization with gradient descent). (b) Olivetti Faces dataset (spectral initialization without gradient descent). (c) PBMC3K RNA dataset (spectral initialization with gradient descent).



(d) Credit Card dataset (spectral initialization without gradient descent). (e) ResNet-18 image embeddings (spectral initialization with gradient descent).

Figure 9: Representative t-SNE embeddings across multiple datasets using the spectral initialization with different optimization settings. The results illustrate how dataset structure and optimization interact with the spectral initialization geometry.

5.7 Discussion: Theory vs Practice

Taken together, these results paint a consistent picture. The primary advantage of spectral initialization lies not in reliably achieving a lower post-optimization KL divergence, since gradient descent substantially closes the gap between methods regardless of starting point, but in two related properties that matter for practical use. First, spectral initializations begin in a substantially lower-cost region of the landscape, providing a geometrically meaningful starting configuration that is directly motivated by the theory of Section 4. Second, they exhibit lower variance across independent runs, producing more reproducible embeddings. This distinction underscores a broader principle: in the nonconvex optimization setting of t-SNE, the geometric quality of the initial embedding is often more consequential than its initial objective value.

6 The spectral problem in UMAP

Eigenmap-based methods minimize the quadratic Dirichlet energy

$$\sum_{i,j} w_{ij} \|y_i - y_j\|^2,$$

whose minimizer lies in the eigenspace of the graph Laplacian. Spectral methods, therefore, apply naturally to algorithms whose objectives are approximately quadratic in pairwise distances (e.g., PCA or the attractive term of t-SNE). UMAP, however, optimizes a binary cross-entropy cost that depends on logarithmic distortions of distances. This leads to forces that are highly nonlinear, asymmetric, and not representable through any Laplacian quadratic form. To illustrate this, we examine the dependence of the UMAP cost on $\|y_i - y_j\|$.

6.1 UMAP's cost function

The cost function and q_{ij} used in UMAP is,

$$C = \sum_{i \neq j} \left[p_{ij} \log \frac{p_{ij}}{q_{ij}} + (1 - p_{ij}) \log \frac{1 - p_{ij}}{1 - q_{ij}} \right]$$

$$q_{ij} = \frac{1}{1 + a \|y_i - y_j\|^{2b}}$$

For $a = b = 1$ we have

$$q_{ij} = \frac{1}{1 + \|y_i - y_j\|^2}$$

Since $x \log x \rightarrow 0$ as $x \rightarrow 0$, the term $p_{ij} \log(p_{ij}/q_{ij})$ vanishes when $p_{ij} = 0$; however, the second term still remains.

We will consider our cost function:

$$\begin{aligned}
C &= \sum_{i \neq j} \left[p_{ij} \log \frac{p_{ij}}{q_{ij}} + (1 - p_{ij}) \log \frac{1 - p_{ij}}{1 - q_{ij}} \right] \\
&= - \sum_{i \neq j} \left[p_{ij} \log q_{ij} + (1 - p_{ij}) \log(1 - q_{ij}) \right] \\
&= \sum_{i \neq j} \left[p_{ij} \log(1 + \|y_i - y_j\|^2) - (1 - p_{ij}) \log(\|y_i - y_j\|^2) \right. \\
&\quad \left. + (1 - p_{ij}) \log(1 + \|y_i - y_j\|^2) \right] \\
&= \sum_{i \neq j} \left[\log(1 + \|y_i - y_j\|^2) - (1 - p_{ij}) \log(\|y_i - y_j\|^2) \right]
\end{aligned}$$

Any terms that do not depend on p_{ij} can be ignored, and thus what remains is

$$C = \sum_{i \neq j} p_{ij} \log(\|y_i - y_j\|^2) \not\approx \sum_{i \neq j} p_{ij} \|y_i - y_j\|^2$$

Unlike the quadratic Dirichlet energy minimized by eigenmap methods, the logarithmic term gives UMAP an attraction force that diverges as points get close. This nonlinearity destroys any linear spectral structure, which is why, from a theoretical perspective, no graph Laplacian eigenproblem can approximate the UMAP embedding.

7 Conclusion

This thesis has investigated the mathematical relationship between spectral graph theory and the t-SNE algorithm, with the goal of providing a principled, theoretically grounded initialization strategy. The central contribution is a proof that, in the small-scale regime, the SNE cost function admits a quadratic approximation whose minimizers are precisely the eigenvectors of the matrix $D^{(p)} - P$. This establishes a precise mathematical connection between spectral embeddings and t-SNE, one that justifies the use of spectral initialization not merely as a heuristic, but as a structurally motivated starting point aligned with the dominant term of the objective. Empirically, this alignment produces more consistent convergence and reduced variance across runs, with the primary advantage lying in the quality of the starting configuration rather than in reliably lower final KL divergence after optimization.

7.1 Limitations

Two principal limitations bound the scope of the present study. First, the analysis was limited to moderate-sized datasets. Spectral methods require

eigen-decompositions of $n \times n$ matrices, which scale as $O(n^3)$ in the worst case and become impractical for large n . This restricts direct applicability to settings where scalable approximations are available.

Second, the experiments used fixed hyperparameters across all initialization strategies, which means the results do not account for potential interactions between initialization and tuning, a factor that may affect the magnitude of the observed advantages. A spectral starting point, being closer to a good minimum, may require less aggressive early exaggeration or benefit from a different learning rate schedule. Studying these interactions could yield practical guidance for practitioners deploying spectral initialization.

7.2 Broader Implications

Understanding the role of initialization contributes to a more principled view of dimensionality reduction as an optimization problem over geometric representations. This matters beyond the theoretical, since t-SNE and related methods are now standard tools in genomics, neuroscience, and clinical research, where embedding quality directly affects scientific conclusions. A poorly organized embedding can cause researchers to misidentify cell populations, misread developmental trajectories in single-cell data, or overlook meaningful structure in high-dimensional clinical records. Initialization strategies grounded in theory, rather than in computational convenience, reduce the risk of such errors and make embeddings more reproducible across runs.

More broadly, this work illustrates a recurring theme in modern machine learning: nonlinear methods often contain latent linear structure that becomes visible under the right limiting or approximating lens. The quadratic approximation of the SNE objective is one instance of this phenomenon. Identifying such a structure does more than explain existing practice, since it provides a language in which to formulate better algorithms, derive convergence guarantees, and reason about what properties of the data a method is actually sensitive to. The connection established here between the small-scale limit of the SNE cost and the eigenvector problem for $D^{(p)} - P$ is, in this sense, not merely a justification for a practical heuristic, but a small step toward a more unified mathematical theory of nonlinear dimensionality reduction.

References

- [1] Mikhail Belkin and Partha Niyogi. Laplacian Eigenmaps and Spectral Techniques for Embedding and Clustering. In *Advances in Neural Information Processing Systems*, vol. 15, pp. 585–591, 2003. <https://www2.imm.dtu.dk/projects/manifold/Papers/Laplacian.pdf>.
- [2] Kaggle Community. CIFAR-10 Dataset. <https://www.kaggle.com/competitions/cifar-10/data?select=trainLabels.csv>. Accessed: December 2, 2025.
- [3] Kaggle Community. Credit Card Fraud Detection Dataset. <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>. Accessed: December 2, 2025.
- [4] Laurens van der Maaten and Geoffrey Hinton. Visualizing High-Dimensional Data Using t-SNE. *Journal of Machine Learning Research*, 9:2579–2605, 2008. <https://www.jmlr.org/papers/volume9/vandermaaten08a/vandermaaten08a.pdf>.
- [5] Claude E. Shannon. *A Mathematical Theory of Communication*. Bell System Technical Journal, 27(3–4):379–423, 623–656, 1948.
- [6] Ben Weinkove. *Stochastic Neighborhood Embedding and the Gradient Flow of Relative Entropy*. arXiv preprint arXiv:2409.16963, 2024. <https://arxiv.org/pdf/2409.16963>.

```

# Import necessary libraries
import os
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.datasets import mnist
import scipy.io
from PIL import Image
import random
from scipy.sparse import csr_matrix
from scipy.sparse.linalg import eigsh
from sklearn.neighbors import NearestNeighbors
from sklearn.metrics import pairwise_distances

def grid_search(diff_i: np.ndarray, i: int, perplexity: int) -> float:
    """
    Helper function to obtain sigma's based on user-specified perplexity.

    Parameters:
    diff_i (np.ndarray): Array containing the pairwise differences
        between data points.
    i (int): Index of the current data point.
    perplexity (int): User-specified perplexity value.

    Returns:
    float: The value of sigma that satisfies the perplexity condition.
    """

    result = np.inf # Set first result to be infinity

    norm = np.linalg.norm(diff_i, axis=1)
    std_norm = np.std(norm) # Use standard deviation of norms to define
        search space

    for sigma_search in np.linspace(0.01 * std_norm, 5 * std_norm, 200):
        # Equation 1 Numerator
        p = np.exp(-(norm**2) / (2 * sigma_search**2))

        # Set p = 0 when i = j
        p[i] = 0

        # Equation 1 (epsilon -> 0)
        epsilon = np.nextafter(0, 1)
        p_new = np.maximum(p / (np.sum(p)+epsilon), epsilon)

        # Shannon Entropy
        H = -np.sum(p_new * np.log2(p_new))

        # Get log(perplexity equation) as close to equality
        if np.abs(np.log(perplexity) - H * np.log(2)) < np.abs(result):

```

```

        result = np.log(perplexity) - H * np.log(2)
        sigma = sigma_search

    return sigma

def get_original_pairwise_affinities(X: np.ndarray, perplexity: int = 10)
    -> np.ndarray:
    """
    Function to obtain affinities matrix.

    Parameters:
    X (np.ndarray): The input data array.
    perplexity (int): The perplexity value for the grid search.

    Returns:
    np.ndarray: The pairwise affinities matrix.
    """

    n = len(X)

    print("Computing Pairwise Affinities...")

    p_ij = np.zeros(shape=(n, n))
    for i in range(0, n):
        # Equation 1 numerator
        diff = X[i] - X
        sigma_i = grid_search(diff, i, perplexity) # Grid Search for sigma_i
        norm = np.linalg.norm(diff, axis=1)
        p_ij[i, :] = np.exp(-(norm**2) / (2 * sigma_i**2))

        # Set p = 0 when j = i
        np.fill_diagonal(p_ij, 0)

        # Equation 1
        p_ij[i, :] = p_ij[i, :] / np.sum(p_ij[i, :])

    # Set 0 values to minimum numpy value (epsilon approx. = 0)
    epsilon = np.nextafter(0, 1)
    p_ij = np.maximum(p_ij, epsilon)

    print("Completed Pairwise Affinities Matrix.\n")

    return p_ij

def get_symmetric_p_ij(p_ij: np.ndarray) -> np.ndarray:
    """
    Function to obtain symmetric affinities matrix utilized in t-SNE.

    Parameters:
    p_ij (np.ndarray): The input affinity matrix.

```

```

    """Returns:
    np.ndarray: The symmetric affinities matrix.

    """
    print("Computing Symmetric p_ij matrix...")

    n = len(p_ij)
    p_ij_symmetric = np.zeros(shape=(n, n))
    for i in range(0, n):
        for j in range(0, n):
            p_ij_symmetric[i, j] = (p_ij[i, j] + p_ij[j, i]) / (2 * n)

    # Set 0 values to minimum numpy value (epsilon approx. = 0)
    epsilon = np.nextafter(0, 1)
    p_ij_symmetric = np.maximum(p_ij_symmetric, epsilon)

    print("Completed Symmetric p_ij Matrix.\n")

    return p_ij_symmetric

def spectral_init(X: np.ndarray, n_dimensions: int = 2, k: int = 10) -> np.
    ndarray:
    """
    Spectral initialization for t-SNE using the first non-trivial
    eigenvectors
    of the unnormalized graph Laplacian.

    Parameters:
    X (np.ndarray): Input data (n_samples x n_features)
    n_dimensions (int): Target number of dimensions
    k (int): Number of nearest neighbors for graph construction

    Returns:
    np.ndarray: Initial embedding (n_samples x n_dimensions)
    """
    n = X.shape[0]

    p_ij = get_original_pairwise_affinities(X, 30)
    p_ij_symmetric = get_symmetric_p_ij(p_ij)

    W = p_ij_symmetric.copy()
    np.fill_diagonal(W, 0)

    D = np.diag(W.sum(axis=1))
    L = D - W

    eigvals, eigvecs = np.linalg.eigh(L)
    return eigvecs[:, 1:n_dimensions + 1]

def laplacian_eigenmaps_init(X: np.ndarray, n_dimensions: int = 2, k: int =

```

```

    10, sigma: float = None, normalized: bool = True) -> np.ndarray:
    """
    Laplacian Eigenmaps initialization for t-SNE Using Belkin and Niyogis
    definition.

    Parameters:
    X: np.ndarray, shape (n_samples, n_features)
    Input data.
    n_dimensions: int
    Number of target dimensions.
    k: int
    Number of nearest neighbors to use when constructing the graph.
    sigma: float or None
    Heat-kernel width. If None, set to the average distance to the k-th
    neighbor.
    normalized: bool
    If True, use symmetric normalized laplacian  $L = I - D^{-1/2} * W * D^{-1/2}$ 
    If False, use unnormalized Laplacian  $L = D - W$ 

    Returns:
    Y: np.ndarray, shape (n_samples, n_dimensions)
    The embedding given by the nontrivial eigenvectors of L
    """
    n = X.shape[0]

    # --- Step 1: kNN graph (unchanged) ---
    nbrs = NearestNeighbors(n_neighbors=k+1, algorithm='auto').fit(X)
    distances, indices = nbrs.kneighbors()
    distances, indices = distances[:, 1:], indices[:, 1:]

    # --- Step 2: Weights (FIX 1 & 2) ---
    rows = np.repeat(np.arange(n), k)
    cols = indices.ravel()

    if sigma is None:
        # Simple 0-1 weights (B&N Section 3.1)
        vals = np.ones(len(rows))
    else:
        # Heat kernel with fixed t, not data-adaptive sigma (FIX 1)
        vals = np.exp(-distances.ravel()*2 / sigma)

    W = csr_matrix((vals, (rows, cols)), shape=(n, n))
    W = W.maximum(W.T) # Union of neighborhoods, not average (FIX 2)

    # --- Step 3: Laplacian (FIX 3) ---
    degrees = np.array(W.sum(axis=1)).ravel()

    if normalized:
        inv_sqrt_deg = np.zeros_like(degrees)
        nonzero = degrees > 0

```

```

        inv_sqrt_deg[nonzero] = 1.0 / np.sqrt(degrees[nonzero])
        D_inv_sqrt = csr_matrix(np.diag(inv_sqrt_deg))
        L = csr_matrix(np.eye(n)) - D_inv_sqrt @ W @ D_inv_sqrt
    else:
        # Unnormalized:  $L = D - W$  <-- true B&N default (FIX 3)
        D = csr_matrix(np.diag(degrees))
        L = D - W

    # --- Step 4: Eigenvectors (unchanged) ---
    vals, vecs = eigsh(L, k=n_dimensions+1, which='SM')

    return vecs[:, 1:n_dimensions+1]

def initialization(X: np.ndarray, n_dimensions: int = 2, initialization:
    str = "PCA", **kwargs) -> np.ndarray:
    if initialization == "random":
        return np.random.normal(loc=0, scale=1e-4, size=(len(X),
            n_dimensions))

    elif initialization == "PCA":
        X_centered = X - X.mean(axis=0)
        _, _, Vt = np.linalg.svd(X_centered)
        return X_centered @ Vt.T[:, :n_dimensions]

    elif initialization == "spectral":
        k = kwargs.get("k", 10) # allow user to pass custom k
        return spectral_init(X, n_dimensions=n_dimensions, k=k)

    elif initialization == "laplacian":
        k = kwargs.get("k", 10)
        sigma = kwargs.get("sigma", None)
        return laplacian_eigenmaps_init(X, n_dimensions=n_dimensions, k=k,
            sigma=sigma)

    else:
        raise ValueError("Initialization must be 'random', 'PCA', 'spectral',
            'or' 'laplacian'")

def get_low_dimensional_affinities(Y: np.ndarray) -> np.ndarray:
    """
    Obtain low-dimensional affinities.

    Parameters:
    Y (np.ndarray): The low-dimensional representation of the data points.

    Returns:
    np.ndarray: The low-dimensional affinities matrix.

```

```

"""

n = len(Y)
q_ij = np.zeros(shape=(n, n))

for i in range(0, n):
    # Equation 4 Numerator
    diff = Y[i] - Y
    norm = np.linalg.norm(diff, axis=1)
    q_ij[i, :] = (1 + norm**2) ** (-1)

    # Set p = 0 when j = i
    np.fill_diagonal(q_ij, 0)

    # Equation 4
    q_ij = q_ij / q_ij.sum()

    # Set 0 values to minimum numpy value (epsilon approx. = 0)
    epsilon = np.nextafter(0, 1)
    q_ij = np.maximum(q_ij, epsilon)

return q_ij

def get_gradient(p_ij: np.ndarray, q_ij: np.ndarray, Y: np.ndarray) -> np.
    ndarray:
    """
    Obtain gradient of cost function at current point Y.

    Parameters:
    p_ij (np.ndarray): The joint probability distribution matrix.
    q_ij (np.ndarray): The Student's t-distribution matrix.
    Y (np.ndarray): The current point in the low-dimensional space.

    Returns:
    np.ndarray: The gradient of the cost function at the current point Y.
    """

    n = len(p_ij)

    # Compute gradient
    gradient = np.zeros(shape=(n, Y.shape[1]))
    for i in range(0, n):
        # Equation 5
        diff = Y[i] - Y
        A = np.array([(p_ij[i, :] - q_ij[i, :])])
        B = np.array([(1 + (np.linalg.norm(diff, axis=1)) ** 2) ** (-1)])
        C = diff
        gradient[i] = 4 * np.sum((A * B).T * C, axis=0)

    return gradient

```

```

def tsne_eigen(
    X: np.ndarray,
    perplexity: int = 30,
    T: int = 1000,
    eta: int = 500,
    early_exaggeration: int = 12,
    initialization_method = 'PCA',
    n_dimensions: int = 2,
    **kwargs
) -> np.ndarray:
    """
    t-SNE (t-Distributed Stochastic Neighbor Embedding) algorithm
    implementation.

    Args:
        X (np.ndarray): The input data matrix of shape (n_samples,
            n_features).
        perplexity (int, optional): The perplexity parameter. Default is 10.

        T (int, optional): The number of iterations for optimization.
            Default is 1000.
        Eta (int, optional): The learning rate for updating the low-
            dimensional embeddings. Default is 200.
        early_exaggeration (int, optional): The factor by which the
            pairwise affinities are exaggerated
            during the early iterations of optimization. Default is 4.
        n_dimensions (int, optional): The number of dimensions of the low-
            dimensional embeddings. Default is 2.

    Returns:
        list[np.ndarray, np.ndarray]: A list containing the final low-
            dimensional embeddings and the history
            of embeddings at each iteration.

    """
    n = len(X)

    # Get original affinities matrix
    p_ij = get_original_pairwise_affinities(X, perplexity)
    p_ij_symmetric = get_symmetric_p_ij(p_ij)

    # Initialization
    Y = np.zeros(shape=(T, n, n_dimensions))
    Y_minus1 = np.zeros(shape=(n, n_dimensions))
    Y[0] = Y_minus1
    Y1 = initialization(X, n_dimensions=n_dimensions, initialization=
        initialization_method, **kwargs)
    Y[1] = np.array(Y1)

    print("Optimizing Low Dimensional Embedding...")
    # Optimization

```



```

for t in range(1, T - 1):
    # Momentum & Early Exaggeration
    if t < 250:
        alpha = 0
        early_exaggeration = early_exaggeration
    else:
        alpha = 0
        early_exaggeration = 1

    # Get Low Dimensional Affinities
    q_ij = get_low_dimensional_affinities(Y[t])

    # Get Gradient of Cost Function
    gradient = get_gradient(early_exaggeration * p_ij_symmetric, q_ij, Y
                             [t])

    # Update Rule
    Y[t + 1] = Y[t] - eta * gradient + alpha * (Y[t] - Y[t - 1]) # Use
    negative gradient

    # Compute current value of cost function
    if t % 50 == 0 or t == 1:
        cost = np.sum(p_ij_symmetric * np.log(p_ij_symmetric / q_ij))
        print(f"Iteration_{t}: Value of Cost Function is {cost}")

    print(
        f"Completed Low Dimensional Embedding: Final Value of Cost Function
        is {np.sum(p_ij_symmetric * np.log(p_ij_symmetric / q_ij))}"
    )
    solution = Y[-1]

    return solution

# Load MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Select the first n images
# For simplicity choose n = 1000
x_train = x_train[:1000]
y_train = y_train[:1000]

# Flatten images from 28x28 to 784
x_train_flat = x_train.reshape((x_train.shape[0], -1)).astype(np.float32)
x_test_flat = x_test.reshape((x_test.shape[0], -1)).astype(np.float32)

# Normalize the data
scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train_flat)
x_test_scaled = scaler.transform(x_test_flat)

# Apply PCA with 30 components (standard in academic research)

```

```

pca = PCA(n_components=30)
x_train_pca = pca.fit_transform(x_train_scaled)
x_test_pca = pca.transform(x_test_scaled)

# Example usage:
Y = tsne_eigen(x_train_pca, initialization_method="spectral", T = 1000)

# You can set initialization_method = {random, PCA, spectral, laplacian}
# T is the number of steps of gradient descent.
# If you want to see no gradient descent steps set T=3, otherwise standard
  values are 500, 1000, or 2000.

# To visualize:
plt.scatter(Y[:, 0], Y[:, 1], c=y_train, cmap='tab10')
plt.title('MNIST_t-SNE') # Change the title to fit the initialization
  method and gradient descent size
plt.show()

```